

36. 目的関数重み CEM 学習

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	5
5.3 import 群の詳細	7
6 このファイルを読む前に必要な基礎知識	8
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	8
6.2 名前、代入、型、参照	8
6.3 関数、引数、戻り値、スコープ	9
6.4 module、package、import、console_scripts	9
6.5 例外、try/finally、with、資源解放	9
6.6 class、独自クラス、インスタンス、self、継承	10
6.7 先頭アンダースコア、__init__、名前付けの作法	10
6.8 デコレータと @dataclass	10
6.9 list、dict、deque、NumPy 配列、pandas DataFrame	11
6.10 CLI、PowerShell、Bash、環境変数、終了コード	11
6.11 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明	11
6.12 Cross-Entropy Method を式と実装で理解する	12
6.13 制御、状態、入力、モデル予測制御 MPC	12
7 関数・クラスを上から順に解説	13

7.1	L83 クラス TermSpec	13
7.2	L92 クラス ScenarioSpec	14
7.3	L99 関数 read_yaml	14
7.4	L104 関数 write_yaml	15
7.5	L110 関数 ensure_dir	16
7.6	L114 関数 log_trial_to_tensorboard	17
7.7	L147 関数 repo_relative	18
7.8	L159 関数 failed_scenario_result	19
7.9	L217 関数 compile_tex	21
7.10	L234 関数 latex_escape	22
7.11	L253 関数 format_override_value	23
7.12	L265 関数 canonical_runtime_weights	24
7.13	L270 関数 set_nested_value	25
7.14	L283 関数 speed_series	26
7.15	L291 関数 upper_cost_specs	26
7.16	L336 関数 vector_to_weights	28
7.17	L347 関数 mirror_legacy_weights	29
7.18	L360 関数 build_reference_free_profile	30
7.19	L397 関数 default_scenarios	32
7.20	L482 関数 run_single_scenario	34
7.21	L613 関数 evaluate_simulation	37
7.22	L759 関数 aggregate_candidate	41
7.23	L820 関数 run_candidate	43
7.24	L858 関数 save_trial_checkpoint	44
7.25	L868 関数 csv_row_count	45
7.26	L873 関数 summarize_path	46
7.27	L896 関数 dataframe_to_markdown	47
7.28	L910 関数 flatten_scalars	48
7.29	L920 関数 build_current_asset_manifests	49
7.30	L986 関数 render_report	51
7.31	L1138 関数 render_report.tex_path_rel	55
7.32	L1504 関数 main	56
7.33	CLI 引数	60
8	処理の流れ	61
9	このファイルの位置づけ	61

1 対象

項目	内容
ファイル	scripts/tune_upper_planner_weights.py
ソース SHA-256	7fa117bb2a11983fb9101004a230ec22fb899a1c95d928fecb2e4743f7a0b22f
種別	Python
区分	planning research
役割	複数シナリオで upper planner の cost weight を探索し、risk-aware な candidate を評価する。
起動文脈	learn action の中心。

2 このファイルを読む理由

複数シナリオで upper planner の cost weight を探索し、risk-aware な candidate を評価する。

- multi-scenario、CVaR、chance gate を扱う。
- operational release ではなく exact validation 前段の性格が強い。

3 呼び出し関係

3.1 どこから呼ばれるか

- scripts/solar_control.sh

3.2 次に読むと理解が進むファイル

- mpc_solarcar/upper_cost.py
- scripts/solar_sim.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/risk_utils.py
- mpc_solarcar/schedule_utils.py
- mpc_solarcar/upper_cost.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
@dataclass
class TermSpec:
    name: str
    lo: float
```

```

    hi: float
    init_log10: float
    threshold: float = 1.0e-4

@dataclass
class ScenarioSpec:
    name: str
    cfg_overrides: Dict[str, object]
    cli_overrides: Dict[str, object]
    weight: float

def read_yaml(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as f:
        return yaml.safe_load(f) or {}

```

5 主要構造の読み取り

主要クラスは TermSpec, ScenarioSpec。主要関数は read_yaml, write_yaml, ensure_dir, log_trial_to_tensorboard, repo_relative, failed_scenario_result, compile_tex, latex_escape。CLI 引数宣言は 19 件。

5.1 主要クラス・関数

行	種別	名前	補足
83	class	TermSpec	
92	class	ScenarioSpec	
99	function	read_yaml	args: path
104	function	write_yaml	args: path, payload
110	function	ensure_dir	args: path
114	function	log_trial_to_tensorboard	args: writer, prefix, result, step
147	function	repo_relative	args: path_like
159	function	failed_scenario_result	args: scenario_name, scenario_weight, upper_cost
217	function	compile_tex	args: tex_path
234	function	latex_escape	args: text
253	function	format_override_value	args: value
265	function	canonical_runtime_weights	args: weights
270	function	set_nested_value	args: cfg, dotted_key, value
283	function	speed_series	args: df
291	function	upper_cost_specs	args: cfg
336	function	vector_to_weights	args: specs, vec, base_cfg
347	function	mirror_legacy_weights	args: cfg, upper_cost
360	function	build_reference_free_profile	args: profile_yaml, output_dir
397	function	default_scenarios	args: profile_yaml

482	function	run_single_scenario	args: profile_yaml, output_dir, candidate_name, scenario, upper_cost, cfg_overrides, cli_overrides
613	function	evaluate_simulation	args: profile_yaml, summary, sim_df, detail_df, upper_cost
759	function	aggregate_candidate	args: candidate, scenario_results, weights, risk_config
820	function	run_candidate	args: profile_yaml, output_dir, candidate_name, upper_cost, cfg_overrides, cli_overrides, scenarios, risk_config
858	function	save_trial_checkpoint	args: output_dir, trials, best_result
868	function	csv_row_count	args: path
873	function	summarize_path	args: path
896	function	dataframe_to_markdown	args: df
910	function	flatten_scalars	args: prefix, payload, rows
920	function	build_current_asset_manifests	args: profile_yaml, fit_summary, output_dir
986	function	render_report	args: output_dir, source_profile_yaml, eval_profile_yaml, fit_summary, baseline_result, tuned_result, trials_df, best_weights, manifest_paths, removed_reference, specs, search_cfg, tensorboard_dir
1138	function	tex_path_rel	args: path
1504	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
19	matplotlib.use(...) を実行する。
25	例外処理を伴う try ブロックを実行する。
30	ROOT に Path(__file__).resolve().parents[1] の結果を代入する。
31	条件 str(ROOT) not in sys.path を判定し、真なら内部処理を行う。
39	DEFAULT_PROFILE に ROOT / 'project_packages' / 'bwsc2025_fitted_mle4' / 'profile.yaml' の結果を代入する。

42 LITERATURE に [{ 'label': 'de Boer et al. (2005)', 'title': 'A Tutorial on the Cross-Entropy Method', 'url': 'https://doi.org/10.1007/s10479-005-5724-z', 'note': 'CEM is a generic derivative-free method for hard optimization problems.' }, { 'label': 'Gros and Zanon (2019/2020)', 'title': 'Data-driven Economic NMPC using Reinforcement Learning', 'url': 'https://arxiv.org/pdf/1904.04152', 'note': 'RL can tune stage cost, terminal cost, and constraints of MPC/Economic MPC.' }, { 'label': 'Zarrouki et al. (2024)', 'title': 'A Safe Reinforcement Learning driven Weights-varying Model Predictive Controller', 'url': 'https://arxiv.org/pdf/2402.02624', 'note': 'Safe RL can adapt MPC weights within a restricted safe search space.' }, { 'label': 'Howlett et al. (1997)', 'title': 'Optimal driving strategy for a solar car on a level road', 'url': 'https://doi.org/10.1093/imaman/8.1.59', 'note': 'Solar-race strategy is governed by a tight energy-speed trade-off.' }, { 'label': 'Pudney and Howlett (2002)', 'title': 'Critical Speed Control of a Solar Car', 'url': 'https://link.springer.com/article/10.1023/A%3A1020907101234', 'note': 'Large unnecessary speed deviations are undesirable in solar-race operation.' }, { 'label': 'Byrd et al. (1995)', 'title': 'A Limited Memory Algorithm for Bound Constrained Optimization', 'url': 'https://doi.org/10.1137/0916069', 'note': 'L-BFGS-B provides bounded local refinement after global candidate search.' }] の結果を代入する。

83 クラス TermSpec を定義する。

92 クラス ScenarioSpec を定義する。

99 関数 read_yaml を定義する。

104 関数 write_yaml を定義する。

110 関数 ensure_dir を定義する。

114 関数 log_trial_to_tensorboard を定義する。

147 関数 repo_relative を定義する。

159 関数 failed_scenario_result を定義する。

217 関数 compile_tex を定義する。

234 関数 latex_escape を定義する。

253 関数 format_override_value を定義する。

265 関数 canonical_runtime_weights を定義する。

270 関数 set_nested_value を定義する。

283 関数 speed_series を定義する。

291 関数 upper_cost_specs を定義する。

336 関数 vector_to_weights を定義する。

347 関数 mirror_legacy_weights を定義する。

360 関数 build_reference_free_profile を定義する。

397 関数 default_scenarios を定義する。

482 関数 run_single_scenario を定義する。

613 関数 evaluate_simulation を定義する。

759 関数 aggregate_candidate を定義する。

820 関数 run_candidate を定義する。

858 関数 save_trial_checkpoint を定義する。

868 関数 csv_row_count を定義する。

873 関数 summarize_path を定義する。

896 関数 dataframe_to_markdown を定義する。

910 関数 flatten_scalars を定義する。

920 関数 build_current_asset_manifests を定義する。

986 関数 render_report を定義する。

1504 関数 main を定義する。

1940 条件 __name__ == '__main__' を判定し、真なら内部処理を行う。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
2	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
4	<code>importargparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L1505。
5	<code>importjson</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L571, L607, L852, L1936, L1937。
6	<code>importmath</code>	Python 標準のスカラ・数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L259, L299, L300, L301, L302, L303, L304, L305, ...。
7	<code>importos</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L154, L205, L206, L207, L208, L209, L210, L211, ...。
8	<code>importshutil</code>	shutil モジュールを利用するため。このファイル内での主な使用位置は L1862。
9	<code>importsubprocess</code>	git 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L220, L224, L225, L228, L545, L550。
10	<code>importsys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L31, L32, L504。
11	<code>importtextwrap</code>	textwrap モジュールを利用するため。このファイル内での主な使用位置は L1444。
12	<code>fromdataclassesimportdataclass</code>	dataclasses から dataclass を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L82, L91。
13	<code>fromdatetimeimportdatetime</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L965, L1163, L1563。
14	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L30, L99, L104, L110, L151, L167, L168, L169, ...。
15	<code>fromtypingimportDict, Iterable,List</code>	typing から Dict, Iterable, List を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L94, L95, L114, L162, L165, L166, L174, L265, ...。
17	<code>importmatplotlib</code>	matplotlib モジュールを利用するため。このファイル内での主な使用位置は L19。
20	<code>importmatplotlib. pyplotasplt</code>	matplotlib.pyplot モジュールを利用するため。このファイル内での主な使用位置は L1037, L1038, L1039, L1040, L1041, L1042, L1043, L1044, ...。
21	<code>importnumpyasnp</code>	数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L288, L336, L339, L627, L637, L639, L643, L653, ...。
22	<code>importpandasaspd</code>	CSV や時刻列の読み込み・整形・再サンプリングに使うため。このファイル内での主な使用位置は L283, L288, L572, L573, L616, L617, L623, L626, ...。

23	<code>importyaml</code>	<code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L101, L107。
26	<code>fromtorch.utils.tensorboardimportSummaryWriter</code>	<code>torch.utils.tensorboard</code> から <code>SummaryWriter</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L28, L1567。
34	<code>frommpc_solarcar.schedule_utilsimportDriveSchedule</code>	<code>schedule_utils.py</code> から <code>DriveSchedule</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/schedule_utils.py</code> 。このファイル内での主な使用位置は L635。
35	<code>frommpc_solarcar.risk_utilsimportScenarioRiskConfig, aggregate_scenario_scores</code>	<code>risk_utils.py</code> から <code>ScenarioRiskConfig</code> , <code>aggregate_scenario_scores</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/risk_utils.py</code> 。このファイル内での主な使用位置は L763, L775, L828, L1536。
36	<code>frommpc_solarcar.upper_costimportUpperCostConfig, active_upper_cost_terms, load_upper_cost_config</code>	上位 MPC 目的関数から <code>UpperCostConfig</code> , <code>active_upper_cost_terms</code> , <code>load_upper_cost_config</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/upper_cost.py</code> 。このファイル内での主な使用位置は L175, L292, L336, L684, L1135, L1534, L1552。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘`x = f()`’では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのものの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self.step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 デコレータと @dataclass

‘@名前’は、直後に定義した関数またはクラスを別の関数へ渡し、その結果で定義名を置き換えるデコレータ構文である。‘@Class’という一般構文があるのではなく、‘@dataclass’など実際のデコレータ名を

書く。

‘@dataclass’は型注釈付きフィールドから‘__init__’、‘__repr__’、比較処理などを自動生成する。物理パラメータや解析結果のように、名前付きデータを一まとまりで運ぶ用途に向く。

```
from dataclasses import dataclass

@dataclass
class State:
    soc: float
    temperature_c: float

state = State(soc=0.8, temperature_c=30.0)
```

自動生成は検証を自動で保証しない。単位、許容範囲、相互依存制約は‘__post_init__’や別の検証関数で確認する必要がある。

根拠資料

- [Python 公式ライブラリ: dataclasses](#)
- [Python 公式チュートリアル: Classes](#)

6.9 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常‘(N,)’、候補集団なら‘(population, N)’となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.10 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。‘argparse’は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS_DOMAIN_ID、RMW_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

6.11 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた sampling と局所最適化を組み合わせる大域最適化法である。有限 Cartesian grid の全列挙は、その grid 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- SciPy 公式: [scipy.optimize.shgo](#)

6.12 Cross-Entropy Method を式と実装で理解する

CEM は候補を生成する確率分布を持ち、良かった elite 候補から分布を更新する反復的な確率最適化である。このリポジトリの `upper_solver.py` は各制御点速度を独立正規分布で生成し、上下限へ clip する。

$$u_i^{(j)} \sim \mathcal{N}(\mu_i^{(g)}, (\sigma_i^{(g)})^2), \quad u_i^{(j)} \leftarrow \text{clip}(u_i^{(j)}, l_i, h_i)$$
$$\mu_i^{(g+1)} = \frac{1}{K} \sum_{j \in \mathcal{E}_g} u_i^{(j)}, \quad \sigma_i^{(g+1)} = \max \left(\text{Std}_{j \in \mathcal{E}_g} u_i^{(j)}, 0.05(h_i - l_i) \right)$$

ここで $\mathcal{E}_g$ は cost が小さい上位 K 候補である。平均は良い領域へ移り、標準偏差は探索幅を表す。標準偏差の下限は探索が完全に潰れることを避ける。

現行 `hybrid_bounded_minimize` は、`deterministic seed` を評価し、上位候補を L-BFGS-B で局所 refine し、設定と seed 間不一致に応じて CEM を実行し、最後に再度局所 refine する。したがって CEM 単独ではなく hybrid solver である。

CEM で落とした候補を永久保存しないこと自体は通常最適化として自然だが、off-nominal 状態からの再利用には別の policy library 設計が必要である。状態を無制限に全組合せ保存する代わりに、SoC、進捗、時刻、予報誤差、停止状態などの scenario を設計し、近傍 policy を検索して MPC で再最適化する。

根拠資料

- Rubinstein and Kroese: [The Cross-Entropy Method](#)
- SciPy 公式: [scipy.optimize.minimize](#)

6.13 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を $\mathbf{x}$ 、操作入力を $\mathbf{u}$ 、外乱・予報を $\mathbf{w}$ とすると、離散モデルは ' $\mathbf{x}[k+1] = \mathbf{f}(\mathbf{x}[k], \mathbf{u}[k], \mathbf{w}[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L83 クラス TermSpec

- 行範囲: L83–L88
- 所属: module 直下
- 定義: `TermSpec(bases=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- `@` で始まる行は、定義したクラスを加工する `decorator` である。

上から順の処理

1. `name` にを代入する。
2. `lo` にを代入する。
3. `hi` にを代入する。
4. `init_log10` にを代入する。
5. `threshold` に `0.0001` を代入する。

代表コード断片

```
class TermSpec:
    name: str
    lo: float
    hi: float
    init_log10: float
    threshold: float = 1.0e-4
```

7.2 L92 クラス ScenarioSpec

- 行範囲: L92–L96
- 所属: module 直下
- 定義: `ScenarioSpec(bases=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- `@` で始まる行は、定義したクラスを加工する `decorator` である。

上から順の処理

1. `name` にを代入する。
2. `cfg_overrides` にを代入する。
3. `cli_overrides` にを代入する。
4. `weight` にを代入する。

代表コード断片

```
class ScenarioSpec:
    name: str
    cfg_overrides: Dict[str, object]
    cli_overrides: Dict[str, object]
    weight: float
```

7.3 L99 関数 read_yaml

- 行範囲: L99–L101
- 所属: module 直下
- 定義: `read_yaml(path:Path)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `open`, `safe_load`
- 戻り値の要点: `yaml.safe_load(f)` or `{}`

- 主なローカル代入名: f
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. with 文で path.open('r', encoding='utf-8') を管理しながら処理する。
2. yaml.safe_load(f) or {} を返す。

代表コード断片

```
def read_yaml(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as f:
        return yaml.safe_load(f) or {}
```

7.4 L104 関数 write_yaml

- 行範囲: L104–L107
- 所属: module 直下
- 定義: write_yaml(path:Path,payload:dict)->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: mkdir, open, safe_dump
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: f
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `path.parent.mkdir(...)` を実行する。
2. `with` 文で `path.open('w', encoding='utf-8', newline='\n')` を管理しながら処理する。
3. `yaml.safe_dump(...)` を実行する。

代表コード断片

```
def write_yaml(path: Path, payload: dict) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8", newline="\n") as f:
        yaml.safe_dump(payload, f, sort_keys=False, allow_unicode=True)
```

7.5 L110 関数 `ensure_dir`

- 行範囲: L110–L111
- 所属: module 直下
- 定義: `ensure_dir(path:Path)->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `mkdir`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `path.mkdir(...)` を実行する。

代表コード断片

```
def ensure_dir(path: Path) -> None:
    path.mkdir(parents=True, exist_ok=True)
```


7.6 L114 関数 `log_trial_to_tensorboard`

- 行範囲: L114–L144
- 所属: module 直下
- 定義: `log_trial_to_tensorboard(writer, prefix:str, result:Dict[str, object], step:int) ->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `add_scalar`, `float`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `key`, `scalar_keys`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `writer is None` を判定し、真なら内部処理を行う。
2. を返す。
3. `scalar_keys` に `['score', 'score_mean', 'score_worst', 'score_variance', 'loss_cvar', 'scenario_failure_probability', 'chance_constraint_pass', 'final_distance_km', 'final_distance_worst_km', 'avg_speed_kmh', 'min_soc', 'final_soc', 'oscillation_mean_abs_dv_kmh', 'oscillation_p95_abs_dv_kmh', 'current_rms_a', 'pack_slew_rms_kw', 'daylight_stop_h', 'daylight_full_soc_h', 'unused_finish_soc', 'cpu_sec', 'active_term_count']` の結果を代入する。
4. `scalar_keys` を順に走査し、各要素を `key` に入れて処理する。
5. 例外処理を伴う `try` ブロックを実行する。
6. `writer.add_scalar(...)` を実行する。
7. `Exception` を捕捉した場合:
8. `Continue` 文を実行する。

代表コード断片

```
def log_trial_to_tensorboard(writer, prefix: str, result: Dict[str, object], step: int) ->
None:
    if writer is None:
        return
    scalar_keys = [
        "score",
        "score_mean",
        "score_worst",
        "score_variance",
        "loss_cvar",
        "scenario_failure_probability",
        "chance_constraint_pass",
        "final_distance_km",
        "final_distance_worst_km",
        "avg_speed_kmh",
        "min_soc",
        "final_soc",
        "oscillation_mean_abs_dv_kmh",
        "oscillation_p95_abs_dv_kmh",
        "current_rms_a",
        "pack_slew_rms_kw",
        "daylight_stop_h",
        "daylight_full_soc_h",
        "unused_finish_soc",
        "cpu_sec",
        "active_term_count",
    ]
    for key in scalar_keys:
        try:
            writer.add_scalar(f"{prefix}/{key}", float(result[key]), step)
        except Exception:
            continue
```

7.7 L147 関数 repo_relative

- 行範囲: L147–L156
- 所属: module 直下
- 定義: `repo_relative(path_like)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `fspath`, `is_absolute`, `relative_to`, `replace`, `resolve`, `str`, `strip`
- 戻り値の要点: `"/os.fspath(resolved.relative_to(ROOT)).replace('\', '/')/raw.replace('\', '/')`
- 主なローカル代入名: `path`, `raw`, `resolved`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. raw に `str(path_like or "").strip()` の結果を代入する。
2. 条件 `not raw` を判定し、真なら内部処理を行う。
3. ” を返す。
4. path に `Path(raw)` の結果を代入する。
5. 例外処理を伴う try ブロックを実行する。
6. resolved に `path.resolve() if path.is_absolute() else (ROOT / path).resolve()` の結果を代入する。
7. `os.fspath(resolved.relative_to(ROOT)).replace('\\', '/')` を返す。
8. Exception を捕捉した場合:
9. `raw.replace('\\', '/')` を返す。

代表コード断片

```
def repo_relative(path_like) -> str:
    raw = str(path_like or "").strip()
    if not raw:
        return ""
    path = Path(raw)
    try:
        resolved = path.resolve() if path.is_absolute() else (ROOT / path).resolve()
        return os.fspath(resolved.relative_to(ROOT)).replace("\\", "/")
    except Exception:
        return raw.replace("\\", "/")
```

7.8 L159 関数 failed_scenario_result

- 行範囲: L159–L214
- 所属: module 直下
- 定義: `failed_scenario_result(scenario_name:str, scenario_weight:float, upper_cost:Dict[str,float], *, error:str, cfg_overrides:Dict[str,object], cli_overrides:Dict[str,object], summary_json:Path, out_csv:Path, detail_csv:Path, plan_csv:Path, report_html:Path, resolved_yaml:Path, sim_log_path:Path) -> Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `UpperCostConfig`, `active_upper_cost_terms`, `float`, `fspath`, `int`, `len`
- 戻り値の要点: `{'score': -10000000000.0, 'final_distance_km': 0.0, 'avg_speed_kmh': 0.0, 'min_soc': 0.0, 'final_soc': 0.0, 'elapsed_hours': 0.0, 'cpu_sec': 0.0, 'finish_reached': False, 'model_validation_gate_pass': False, 'scenario_feasible': False, 'scenario_feasibility_checks': {'simulation_completed': False}, 'scenario_infeasibility_reasons': [error], 'oscillation_mean_abs_dv_kmh': 1000000.0, 'oscillation_p95_abs_dv_kmh': 1000000.0, 'current_rms_a': 1000000.0, 'pack_slew_rms_kw': 1000000.0, 'high_speed_h': 1000000.0, 'daylight_stop_h': 1000000.0, 'daylight_full_soc_h': 1000000.0, 'unused_finish_soc': 1.0, 'finish_soc_target':`

```
0.0, 'terminal_soc_error': 1.0, 'active_term_count': int(len(active_terms)), 'active_terms': active_terms,
'scenario': scenario_name, 'scenario_weight': float(scenario_weight), 'cfg_overrides': cfg_overrides,
'cli_overrides': cli_overrides, 'summary_json': os.fspath(summary_json), 'out_csv': os.fspath(out_csv),
'detail_csv': os.fspath(detail_csv), 'plan_csv': os.fspath(plan_csv), 'report_html': os.fspath(report_html),
'resolved_yaml': os.fspath(resolved_yaml), 'simulation_log': os.fspath(sim_log_path), 'failed': True, 'error':
error}
```

- 主なローカル代入名: active_terms
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. active_terms に active_upper_cost_terms(UpperCostConfig(**upper_cost), threshold=0.0001) の結果を代入する。
2. {'score': -1000000000.0, 'final_distance_km': 0.0, 'avg_speed_kmh': 0.0, 'min_soc': 0.0, 'final_soc': 0.0, 'elapsed_hours': 0.0, 'cpu_sec': 0.0, 'finish_reached': False, 'model_validation_gate_pass': False, 'scenario_feasible': False, 'scenario_feasibility_checks': {'simulation_completed': False}, 'scenario_infeasibility_reasons': [error], 'oscillation_mean_abs_dv_kmh': 1000000.0, 'oscillation_p95_abs_dv_kmh': 1000000.0, 'current_rms_a': 1000000.0, 'pack_slew_rms_kw': 1000000.0, 'high_speed_h': 1000000.0, 'daylight_stop_h': 1000000.0, 'daylight_full_soc_h': 1000000.0, 'unused_finish_soc': 1.0, 'finish_soc_target': 0.0, 'terminal_soc_error': 1.0, 'active_term_count': int(len(active_terms)), 'active_terms': active_terms, 'scenario': scenario_name, 'scenario_weight': float(scenario_weight), 'cfg_overrides': cfg_overrides, 'cli_overrides': cli_overrides, 'summary_json': os.fspath(summary_json), 'out_csv': os.fspath(out_csv), 'detail_csv': os.fspath(detail_csv), 'plan_csv': os.fspath(plan_csv), 'report_html': os.fspath(report_html), 'resolved_yaml': os.fspath(resolved_yaml), 'simulation_log': os.fspath(sim_log_path), 'failed': True, 'error': error} を返す。

代表コード断片

```
def failed_scenario_result(
    scenario_name: str,
    scenario_weight: float,
    upper_cost: Dict[str, float],
    *,
    error: str,
    cfg_overrides: Dict[str, object],
    cli_overrides: Dict[str, object],
    summary_json: Path,
    out_csv: Path,
    detail_csv: Path,
    plan_csv: Path,
    report_html: Path,
    resolved_yaml: Path,
    sim_log_path: Path,
) -> Dict[str, object]:
    active_terms = active_upper_cost_terms(UpperCostConfig(**upper_cost), threshold=1.0e-4)
    return {
        "score": -1.0e9,
        "final_distance_km": 0.0,
```

```

    "avg_speed_kmh": 0.0,
    "min_soc": 0.0,
    "final_soc": 0.0,
    "elapsed_hours": 0.0,
    "cpu_sec": 0.0,
    "finish_reached": False,
    "model_validation_gate_pass": False,
    "scenario_feasible": False,
    "scenario_feasibility_checks": {"simulation_completed": False},
    "scenario_infeasibility_reasons": [error],
    "oscillation_mean_abs_dv_kmh": 1.0e6,
    "oscillation_p95_abs_dv_kmh": 1.0e6,
    "current_rms_a": 1.0e6,
    "pack_slew_rms_kw": 1.0e6,
    "high_speed_h": 1.0e6,
    ...

```

7.9 L217 関数 compile_tex

- 行範囲: L217–L231
- 所属: module 直下
- 定義: `compile_tex(tex_path:Path)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `CalledProcessError`, `FileNotFoundError`, `exists`, `range`, `run`, `with_suffix`
- 戻り値の要点: `pdf_path`
- 主なローカル代入名: `_`, `pdf_path`, `res`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `FileNotFoundError(pdf_path)`, `subprocess.CalledProcessError(res.returncode, res.args)`
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `pdf_path` に `tex_path.with_suffix('.pdf')` の結果を代入する。
2. `range(2)` を順に走査し、各要素を `_` に入れて処理する。
3. `res` に `subprocess.run(['xelatex', '-interaction=nonstopmode', tex_path.name], cwd=tex_path.parent, check=False, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)` の結果を代入する。
4. 条件 `res.returncode != 0 and (not pdf_path.exists())` を判定し、真なら内部処理を行う。
5. `subprocess.CalledProcessError(res.returncode, res.args)` を送出する。

6. 条件 `not pdf_path.exists()` を判定し、真なら内部処理を行う。
7. `FileNotFoundError(pdf_path)` を送出する。
8. `pdf_path` を返す。

代表コード断片

```
def compile_tex(tex_path: Path) -> Path:
    pdf_path = tex_path.with_suffix(".pdf")
    for _ in range(2):
        res = subprocess.run(
            ["xelatex", "-interaction=nonstopmode", tex_path.name],
            cwd=tex_path.parent,
            check=False,
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
        )
        if res.returncode != 0 and not pdf_path.exists():
            raise subprocess.CalledProcessError(res.returncode, res.args)
    if not pdf_path.exists():
        raise FileNotFoundError(pdf_path)
    return pdf_path
```

7.10 L234 関数 `latex_escape`

- 行範囲: L234–L250
- 所属: module 直下
- 定義: `latex_escape(text:str)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `items`, `replace`, `str`
- 戻り値の要点: `out`
- 主なローカル代入名: `dst`, `out`, `repl`, `src`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `repl` に `{'\\': '\\textbackslash{ }', '&': '\\&', '%': '\\%', '$': '\\$', '#': '\\#', '_': '\\_', '{': '\\{', '}': '\\}', '~': '\\textasciitilde{ }', '^': '\\textasciicircum{ }'}` の結果を代入する。
2. `out` に `str(text)` の結果を代入する。
3. `repl.items()` を順に走査し、各要素を `(src, dst)` に入れて処理する。
4. `out` に `out.replace(src, dst)` の結果を代入する。
5. `out` を返す。

代表コード断片

```
def latex_escape(text: str) -> str:
    repl = {
        "\\": r"\textbackslash{}",
        "&": r"&",
        "%": r"%",
        "$": r"$",
        "#": r"#",
        "_": r"\_",
        "{": r"\{",
        "}": r"\}",
        "~": r"\textasciitilde{}",
        "^": r"\textasciicircum{}",
    }
    out = str(text)
    for src, dst in repl.items():
        out = out.replace(src, dst)
    return out
```

7.11 L253 関数 `format_override_value`

- 行範囲: L253–L262
- 所属: module 直下
- 定義: `format_override_value(value)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `isfinite`, `isinstance`, `str`
- 戻り値の要点: `str(value) / 'true' if value else 'false' / str(value) / '0'`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `isinstance(value, bool)` を判定し、真なら内部処理を行う。
2. `'true' if value else 'false'` を返す。
3. 条件 `isinstance(value, int)` を判定し、真なら内部処理を行う。
4. `str(value)` を返す。
5. 条件 `isinstance(value, float)` を判定し、真なら内部処理を行う。
6. 条件 `math.isfinite(value)` を判定し、真なら内部処理を行う。

7. `f"{value:.12g}"` を返す。
8. `'0'` を返す。
9. `str(value)` を返す。

代表コード断片

```
def format_override_value(value) -> str:
    if isinstance(value, bool):
        return "true" if value else "false"
    if isinstance(value, int):
        return str(value)
    if isinstance(value, float):
        if math.isfinite(value):
            return f"{value:.12g}"
        return "0"
    return str(value)
```

7.12 L265 関数 `canonical_runtime_weights`

- 行範囲: L265–L267
- 所属: module 直下
- 定義: `canonical_runtime_weights(weights: Dict[str, float]) -> Dict[str, float]`
- docstring: Return the exact numeric values serialized into solar_sim overrides.
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `format_override_value`, `items`, `str`
- 戻り値の要点: `{str(key): float(format_override_value(float(value))) for key, value in weights.items()}`
- 主なローカル代入名: `key`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `{str(key): float(format_override_value(float(value))) for key, value in weights.items()}` を返す。

代表コード断片

```
def canonical_runtime_weights(weights: Dict[str, float]) -> Dict[str, float]:
    """Return the exact numeric values serialized into solar_sim overrides."""
    return {str(key): float(format_override_value(float(value))) for key, value in weights.items()}
```


7.13 L270 関数 `set_nested_value`

- 行範囲: L270–L280
- 所属: module 直下
- 定義: `set_nested_value(cfg:dict,dotted_key:str,value)->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `get`, `isinstance`, `split`, `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `child`, `part`, `parts`, `target`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `target` に `cfg` の結果を代入する。
2. `parts` に `[part for part in str(dotted_key).split('.') if part]` の結果を代入する。
3. `parts[:-1]` を順に走査し、各要素を `part` に入れて処理する。
4. `child` に `target.get(part)` の結果を代入する。
5. 条件 `not isinstance(child, dict)` を判定し、真なら内部処理を行う。
6. `child` に `{}` の結果を代入する。
7. `target[part]` に `child` の結果を代入する。
8. `target` に `child` の結果を代入する。
9. 条件 `parts` を判定し、真なら内部処理を行う。
10. `target[parts[-1]]` に `value` の結果を代入する。

代表コード断片

```
def set_nested_value(cfg: dict, dotted_key: str, value) -> None:
    target = cfg
    parts = [part for part in str(dotted_key).split(".") if part]
    for part in parts[:-1]:
        child = target.get(part)
        if not isinstance(child, dict):
            child = {}
            target[part] = child
        target = child
    if parts:
        target[parts[-1]] = value
```

7.14 L283 関数 speed_series

- 行範囲: L283–L288
- 所属: module 直下
- 定義: `speed_series(df:pd.DataFrame)->pd.Series`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Series`, `astype`, `len`, `zeros`
- 戻り値の要点: `pd.Series(np.zeros(len(df), dtype=float)) / df['v_exec_kmh'].astype(float) / df['v_cmd_kmh'].astype(float)`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件‘`v_exec_kmh`’ in `df.columns` を判定し、真なら内部処理を行う。
2. `df['v_exec_kmh'].astype(float)` を返す。
3. 条件‘`v_cmd_kmh`’ in `df.columns` を判定し、真なら内部処理を行う。
4. `df['v_cmd_kmh'].astype(float)` を返す。
5. `pd.Series(np.zeros(len(df), dtype=float))` を返す。

代表コード断片

```
def speed_series(df: pd.DataFrame) -> pd.Series:
    if "v_exec_kmh" in df.columns:
        return df["v_exec_kmh"].astype(float)
    if "v_cmd_kmh" in df.columns:
        return df["v_cmd_kmh"].astype(float)
    return pd.Series(np.zeros(len(df), dtype=float))
```

7.15 L291 関数 upper_cost_specs

- 行範囲: L291–L333
- 所属: module 直下
- 定義: `upper_cost_specs(cfg:UpperCostConfig,*,include_progress_terms:bool=False,include_uncertainty_term:bool=True,include_terminal_term:bool=True)->List[TermSpec]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `TermSpec`, `append`, `extend`, `log10`, `max`

- 戻り値の要点: specs
- 主なローカル代入名: specs
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. specs に [TermSpec('w_speed_smooth', -2.0, 3.0, math.log10(max(cfg.w_speed_smooth, 1e-06))), TermSpec('w_speed_limit', -2.0, 3.0, math.log10(max(cfg.w_speed_limit, 1e-06))), TermSpec('w_current_sq', -5.0, 1.0, math.log10(max(cfg.w_current_sq, 1e-06))), TermSpec('w_pack_energy', -5.0, 2.0, math.log10(max(cfg.w_pack_energy, 1e-06))), TermSpec('w_joule_loss', -5.0, 2.0, math.log10(max(cfg.w_joule_loss, 1e-06))), TermSpec('w_aero_energy', -5.0, 2.0, math.log10(max(cfg.w_aero_energy, 1e-06))), TermSpec('w_mech_energy', -5.0, 2.0, math.log10(max(cfg.w_mech_energy, 1e-06))), TermSpec('w_kinetic_pos', -5.0, 2.0, math.log10(max(cfg.w_kinetic_pos, 1e-06))), TermSpec('w_pack_power_slew', -4.0, 4.0, math.log10(max(cfg.w_pack_power_slew, 1e-06))), TermSpec('w_speed_quartic', -6.0, 2.0, math.log10(max(cfg.w_speed_quartic, 1e-06))), TermSpec('w_solar_headroom', -2.0, 7.0, max(2.0, math.log10(max(cfg.w_solar_headroom, 1e-06)))), TermSpec('w_soc_floor_barrier', -6.0, 4.0, math.log10(max(cfg.w_soc_floor_barrier, 1e-06))), TermSpec('w_terminal_soc_min', -1.0, 4.0, math.log10(max(cfg.w_terminal_soc_min, 1e-06))), TermSpec('w_day_end_soc_min', 2.0, 6.0, math.log10(max(cfg.w_day_end_soc_min, 1e-06)))] の結果を代入する。
2. 条件 include_uncertainty_term を判定し、真なら内部処理を行う。
3. specs.append(...) を実行する。
4. 条件 include_terminal_term を判定し、真なら内部処理を行う。
5. specs.append(...) を実行する。
6. 条件 include_progress_terms を判定し、真なら内部処理を行う。
7. specs.extend(...) を実行する。
8. specs を返す。

代表コード断片

```
def upper_cost_specs(
    cfg: UpperCostConfig,
    *,
    include_progress_terms: bool = False,
    include_uncertainty_term: bool = True,
    include_terminal_term: bool = True,
) -> List[TermSpec]:
    specs = [
        TermSpec("w_speed_smooth", -2.0, 3.0, math.log10(max(cfg.w_speed_smooth, 1.0e-6))),
        TermSpec("w_speed_limit", -2.0, 3.0, math.log10(max(cfg.w_speed_limit, 1.0e-6))),
        TermSpec("w_current_sq", -5.0, 1.0, math.log10(max(cfg.w_current_sq, 1.0e-6))),
        TermSpec("w_pack_energy", -5.0, 2.0, math.log10(max(cfg.w_pack_energy, 1.0e-6))),
        TermSpec("w_joule_loss", -5.0, 2.0, math.log10(max(cfg.w_joule_loss, 1.0e-6))),
        TermSpec("w_aero_energy", -5.0, 2.0, math.log10(max(cfg.w_aero_energy, 1.0e-6))),
```

```

    TermSpec("w_mech_energy", -5.0, 2.0, math.log10(max(cfg.w_mech_energy, 1.0e-6))),
    TermSpec("w_kinetic_pos", -5.0, 2.0, math.log10(max(cfg.w_kinetic_pos, 1.0e-6))),
    TermSpec("w_pack_power_slew", -4.0, 4.0, math.log10(max(cfg.w_pack_power_slew, 1.0e
-6))),
    TermSpec("w_speed_quartic", -6.0, 2.0, math.log10(max(cfg.w_speed_quartic, 1.0e-6)))
,
    # At high SoC the headroom term is multiplied by a small squared SoC
    # excess. Its useful scale is therefore much larger than the legacy
    # 1e-4 seed; allow CEM to explore a physically meaningful range.
    TermSpec("w_solar_headroom", -2.0, 7.0, max(2.0, math.log10(max(cfg.w_solar_headroom
, 1.0e-6)))),
    TermSpec("w_soc_floor_barrier", -6.0, 4.0, math.log10(max(cfg.w_soc_floor_barrier,
1.0e-6))),
    TermSpec("w_terminal_soc_min", -1.0, 4.0, math.log10(max(cfg.w_terminal_soc_min, 1.0
e-6))),
    TermSpec("w_day_end_soc_min", 2.0, 6.0, math.log10(max(cfg.w_day_end_soc_min, 1.0e
-6))),
]
if include_uncertainty_term:
    specs.append(TermSpec("w_uncertainty_reserve", -6.0, 5.0, math.log10(max(cfg.
w_uncertainty_reserve, 1.0e-6))))
if include_terminal_term:
    specs.append(TermSpec("w_soc_terminal", -6.0, 6.0, math.log10(max(cfg.w_soc_terminal
, 1.0e-6))))
if include_progress_terms:
    specs.extend(
        [
            TermSpec("w_progress_lag", -6.0, 4.0, math.log10(max(cfg.w_progress_lag, 1.0
e-6))),
            TermSpec(
...

```

7.16 L336 関数 `vector_to_weights`

- 行範囲: L336–L344
- 所属: module 直下
- 定義: `vector_to_weights(specs:List[TermSpec],vec:np.ndarray,base_cfg:UpperCostConfig)->Dict[str,float]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clip`, `float`, `to_dict`, `zip`
- 戻り値の要点: `weights`
- 主なローカル代入名: `logv`, `raw`, `spec`, `value`, `weights`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `weights` に `base_cfg.to_dict()` の結果を代入する。
2. `zip(specs, vec)` を順に走査し、各要素を `(spec, raw)` に入れて処理する。
3. `logv` に `float(np.clip(raw, spec.lo, spec.hi))` の結果を代入する。
4. `value` に `10.0 ** logv` の結果を代入する。
5. 条件 `value < spec.threshold` を判定し、真なら内部処理を行う。
6. `value` に `0.0` の結果を代入する。
7. `weights[spec.name]` に `float(value)` の結果を代入する。
8. `weights` を返す。

代表コード断片

```
def vector_to_weights(specs: List[TermSpec], vec: np.ndarray, base_cfg: UpperCostConfig) -> Dict[str, float]:
    weights = base_cfg.to_dict()
    for spec, raw in zip(specs, vec):
        logv = float(np.clip(raw, spec.lo, spec.hi))
        value = 10.0 ** logv
        if value < spec.threshold:
            value = 0.0
        weights[spec.name] = float(value)
    return weights
```

7.17 L347 関数 `mirror_legacy_weights`

- 行範囲: L347–L357
- 所属: `module` 直下
- 定義: `mirror_legacy_weights(cfg:dict, upper_cost:Dict[str,float]) ->None`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `get`, `setdefault`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `mpc`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. mpc に `cfg.setdefault('mpc', {})` の結果を代入する。
2. `mpc['upper_cost']` に `upper_cost` の結果を代入する。
3. `mpc['w_dv']` に `float(upper_cost.get('w_speed_smooth', mpc.get('w_dv', 30.0)))` の結果を代入する。
4. `mpc['w_dv_limit']` に `float(upper_cost.get('w_dv_limit', mpc.get('w_dv_limit', 2.0)))` の結果を代入する。
5. `mpc['w_speed_limit']` に `float(upper_cost.get('w_speed_limit', mpc.get('w_speed_limit', 50.0)))` の結果を代入する。
6. `mpc['w_current']` に `float(upper_cost.get('w_current_sq', mpc.get('w_current', 0.01)))` の結果を代入する。
7. `mpc['w_T']` に `float(upper_cost.get('w_temp', mpc.get('w_T', 5.0)))` の結果を代入する。
8. `mpc['w_soc_day_max']` に `float(upper_cost.get('w_soc_day_max', mpc.get('w_soc_day_max', 10000.0)))` の結果を代入する。
9. `mpc['w_soc_day_track']` に `float(upper_cost.get('w_soc_day_track', mpc.get('w_soc_day_track', 0.0)))` の結果を代入する。
10. `mpc['w_soc_terminal']` に `float(upper_cost.get('w_soc_terminal', mpc.get('w_soc_terminal', 0.0)))` の結果を代入する。

代表コード断片

```
def mirror_legacy_weights(cfg: dict, upper_cost: Dict[str, float]) -> None:
    mpc = cfg.setdefault("mpc", {})
    mpc["upper_cost"] = upper_cost
    mpc["w_dv"] = float(upper_cost.get("w_speed_smooth", mpc.get("w_dv", 30.0)))
    mpc["w_dv_limit"] = float(upper_cost.get("w_dv_limit", mpc.get("w_dv_limit", 2.0)))
    mpc["w_speed_limit"] = float(upper_cost.get("w_speed_limit", mpc.get("w_speed_limit",
    50.0)))
    mpc["w_current"] = float(upper_cost.get("w_current_sq", mpc.get("w_current", 0.01)))
    mpc["w_T"] = float(upper_cost.get("w_temp", mpc.get("w_T", 5.0)))
    mpc["w_soc_day_max"] = float(upper_cost.get("w_soc_day_max", mpc.get("w_soc_day_max",
    1.0e4)))
    mpc["w_soc_day_track"] = float(upper_cost.get("w_soc_day_track", mpc.get("
    w_soc_day_track", 0.0)))
    mpc["w_soc_terminal"] = float(upper_cost.get("w_soc_terminal", mpc.get("w_soc_terminal",
    0.0)))
```

7.18 L360 関数 `build_reference_free_profile`

- 行範囲: L360–L394
- 所属: module 直下
- 定義: `build_reference_free_profile(profile_yaml:Path,output_dir:Path,*,disable_uncertainty_reserve:bool=False)->tuple[Path,str]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `append`, `fspath`, `is_absolute`, `isinstance`, `items`, `list`, `pop`, `read_yaml`, `resolve`, `setdefault`, `str`
- 戻り値の要点: `(out_path, removed_reference)`
- 主なローカル代入名: `candidate`, `cfg`, `key`, `meta`, `mpc`, `notes`, `out_path`, `paths`, `ref_cfg`, `removed_reference`, `upper_cost`, `value`

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 7、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `cfg` に `read_yaml(profile_yaml)` の結果を代入する。
2. `removed_reference` に” の結果を代入する。
3. `paths` に `cfg.setdefault('paths', {})` の結果を代入する。
4. 条件 `isinstance(paths, dict)` を判定し、真なら内部処理を行う。
5. `removed_reference` に `str(paths.pop('progress_reference_csv', ''))` の結果を代入する。
6. `list(paths.items())` を順に走査し、各要素を (`key`, `value`) に入れて処理する。
7. 条件 `isinstance(value, str) and value.strip()` を判定し、真なら内部処理を行う。
8. `candidate` に `Path(value)` の結果を代入する。
9. 条件 `not candidate.is_absolute()` を判定し、真なら内部処理を行う。
10. `meta` に `cfg.setdefault('meta', {})` の結果を代入する。
11. `notes` に `meta.setdefault('notes', [])` の結果を代入する。
12. 条件 `isinstance(notes, list)` を判定し、真なら内部処理を行う。
13. `notes.append(...)` を実行する。
14. `mpc` に `cfg.setdefault('mpc', {})` の結果を代入する。
15. `ref_cfg` に `mpc.setdefault('reference_speed_tracking', {})` の結果を代入する。
16. 条件 `isinstance(ref_cfg, dict)` を判定し、真なら内部処理を行う。
17. `ref_cfg['enabled']` に `False` の結果を代入する。
18. `upper_cost` に `mpc.setdefault('upper_cost', {})` の結果を代入する。
19. 条件 `isinstance(upper_cost, dict)` を判定し、真なら内部処理を行う。
20. `upper_cost['w_progress_lag']` に `0.0` の結果を代入する。
21. `upper_cost['w_progress_terminal_lag']` に `0.0` の結果を代入する。
22. 条件 `disable_uncertainty_reserve` を判定し、真なら内部処理を行う。
23. `upper_cost['w_uncertainty_reserve']` に `0.0` の結果を代入する。
24. `upper_cost['reserve_soc_per_hour']` に `0.0` の結果を代入する。
25. `upper_cost['reserve_soc_max_extra']` に `0.0` の結果を代入する。
26. `out_path` に `output_dir / 'self_learning_reference_free_profile.yaml'` の結果を代入する。
27. `write_yaml(...)` を実行する。
28. `(out_path, removed_reference)` を返す。

代表コード断片

```
def build_reference_free_profile(
    profile_yaml: Path,
    output_dir: Path,
    *,
    disable_uncertainty_reserve: bool = False,
) -> tuple[Path, str]:
    cfg = read_yaml(profile_yaml)
    removed_reference = ""
    paths = cfg.setdefault("paths", {})
    if isinstance(paths, dict):
        removed_reference = str(paths.pop("progress_reference_csv", "") or "")
        for key, value in list(paths.items()):
            if isinstance(value, str) and value.strip():
                candidate = Path(value)
                if not candidate.is_absolute():
                    paths[key] = os.fspath((profile_yaml.parent / candidate).resolve())
    meta = cfg.setdefault("meta", {})
    notes = meta.setdefault("notes", [])
    if isinstance(notes, list):
        notes.append("Self-learning tuner generated a reference-free copy for autonomous weight search.")
    mpc = cfg.setdefault("mpc", {})
    ref_cfg = mpc.setdefault("reference_speed_tracking", {})
    if isinstance(ref_cfg, dict):
        ref_cfg["enabled"] = False
    upper_cost = mpc.setdefault("upper_cost", {})
    if isinstance(upper_cost, dict):
        upper_cost["w_progress_lag"] = 0.0
        upper_cost["w_progress_terminal_lag"] = 0.0
        if disable_uncertainty_reserve:
            upper_cost["w_uncertainty_reserve"] = 0.0
            upper_cost["reserve_soc_per_hour"] = 0.0
            upper_cost["reserve_soc_max_extra"] = 0.0
    out_path = output_dir / "self_learning_reference_free_profile.yaml"
    write_yaml(out_path, cfg)
    return out_path, removed_reference
```

7.19 L397 関数 default_scenarios

- 行範囲: L397–L479
- 所属: module 直下
- 定義: `default_scenarios(profile_yaml:Path, *, mode:str='nominal')->List[ScenarioSpec]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ScenarioSpec`, `float`, `get`, `isinstance`, `lower`, `max`, `min`, `read_yaml`, `str`, `strip`
- 戻り値の要点: `[ScenarioSpec('nominal', cfg_overrides={}, cli_overrides={}, weight=0.3), ScenarioSpec('low_solar_high_load', cfg_overrides={'model.P_aux': max(20.0, base_aux + 20.0), 'model.CdA': max(base_cda * 1.1, base_cda + 0.005), 'model.Crr': max(base_crr * 1.05, base_crr + 0.0002)}, cli_overrides={'solar_gain': 0.9, 'poa_gain_drive': 0.94, 'poa_gain_stop': 0.92}, weight=0.2), ScenarioSpec('drag_bias', cfg_overrides={'model.P_aux': max(10.0, base_aux + 10.0), 'model.CdA': max(base_cda * 1.2, base_cda + 0.01), 'model.Crr': max(base_crr * 1.08, base_crr + 0.0004)}, cli_overrides={'solar_gain': 0.97, 'poa_gain_drive': 0.98, 'poa_gain_stop': 0.98}, weight=0.1), ScenarioSpec('low_initial_soc', cfg_overrides={}, cli_overrides={'soc0': max(soc_min + 0.04, base_soc - 0.15)}, weight=0.15), ScenarioSpec('hot_battery', cfg_overrides={}, cli_overrides={'Tb0': min(temp_max_c - 2.0, base_tb_c + 12.0)}, weight=0.1), ScenarioSpec('driver_lag', cfg_overrides={},`


```
cli_overrides={'exec_tau_sec': 4.0, 'exec_reaction_delay_sec': 3.0, 'exec_accel_limit_kmhps': 0.7,
'exec_decel_limit_kmhps': 2.0}, weight=0.1), ScenarioSpec('favorable_weather', cfg_overrides={'model.P_aux':
max(0.0, base_aux - 10.0)}, cli_overrides={'solar_gain': 1.08, 'poa_gain_drive': 1.04, 'poa_gain_stop': 1.04},
weight=0.05)] / [ScenarioSpec('nominal', cfg_overrides={}, cli_overrides={}, weight=1.0)]
```

- 主なローカル代入名: base_aux, base_cda, base_crr, base_soc, base_tb_c, cfg, model, simulation, soc_min, temp_max_c
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `str(mode).strip().lower() == 'nominal'` を判定し、真なら内部処理を行う。
2. `cfg` に `read_yaml(profile_yaml)` の結果を代入する。
3. `model` に `cfg.get('model', {})` if `isinstance(cfg, dict)` else `{}` の結果を代入する。
4. `base_cda` に `float(model.get('CdA', 0.08))` の結果を代入する。
5. `base_crr` に `float(model.get('Crr', 0.008))` の結果を代入する。
6. `base_aux` に `float(model.get('P_aux', 0.0))` の結果を代入する。
7. `simulation` に `cfg.get('simulation', {})` if `isinstance(cfg, dict)` else `{}` の結果を代入する。
8. `base_soc` に `float(simulation.get('soc0', 0.99))` の結果を代入する。
9. `base_tb_c` に `float(simulation.get('Tb0', 30.0))` の結果を代入する。
10. `soc_min` に `float(model.get('soc_min', 0.05))` の結果を代入する。
11. `temp_max_c` に `float(model.get('T_max', 55.0))` の結果を代入する。

`weight=0.05)` を返す。

代表コード断片

```
def default_scenarios(profile_yaml: Path, *, mode: str = "nominal") -> List[ScenarioSpec]:
    if str(mode).strip().lower() == "nominal":
        return [ScenarioSpec("nominal", cfg_overrides={}, cli_overrides={}, weight=1.0)]
    cfg = read_yaml(profile_yaml)
    model = cfg.get("model", {}) if isinstance(cfg, dict) else {}
    base_cda = float(model.get("CdA", 0.08))
    base_crr = float(model.get("Crr", 0.008))
    base_aux = float(model.get("P_aux", 0.0))
    simulation = cfg.get("simulation", {}) if isinstance(cfg, dict) else {}
    base_soc = float(simulation.get("soc0", 0.99))
    base_tb_c = float(simulation.get("Tb0", 30.0))
    soc_min = float(model.get("soc_min", 0.05))
    temp_max_c = float(model.get("T_max", 55.0))
```

```

return [
    ScenarioSpec("nominal", cfg_overrides={}, cli_overrides={}, weight=0.30),
    ScenarioSpec(
        "low_solar_high_load",
        cfg_overrides={
            "model.P_aux": max(20.0, base_aux + 20.0),
            "model.CdA": max(base_cda * 1.10, base_cda + 0.005),
            "model.Crr": max(base_crr * 1.05, base_crr + 0.0002),
        },
        cli_overrides={
            "solar_gain": 0.90,
            "poa_gain_drive": 0.94,
            "poa_gain_stop": 0.92,
        },
        weight=0.20,
    ),
    ScenarioSpec(
        "drag_bias",
        cfg_overrides={
            "model.P_aux": max(10.0, base_aux + 10.0),
            "model.CdA": max(base_cda * 1.20, base_cda + 0.010),
            "model.Crr": max(base_crr * 1.08, base_crr + 0.0004),
        },
        weight=0.20,
    ),
    ...
]

```

7.20 L482 関数 run_single_scenario

- 行範囲: L482–L610
- 所属: module 直下
- 定義: `run_single_scenario(profile_yaml:Path,output_dir:Path,candidate_name:str,scenario:ScenarioSpec,upper_cost:Dict[str,float],cfg_overrides:Dict[str,object],cli_overrides:Dict[str,object])->Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `Path`, `dict`, `dumps`, `ensure_dir`, `evaluate_simulation`, `exists`, `extend`, `failed_scenario_result`, `float`, `format_override_value`, `fspath`
- 戻り値の要点: `result`
- 主なローカル代入名: `cmd`, `detail_csv`, `detail_df`, `key`, `latest_manifest_json`, `log_f`, `merged_cfg`, `merged_cli`, `out_csv`, `plan_csv`, `proc`, `report_html`, `resolved_yaml`, `result`, `scenario_dir`, `sim_df`, `sim_log_path`, `summary`, `summary_json`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 3、`try` 1

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `scenario_dir` に `output_dir / 'candidates' / candidate_name / scenario.name` の結果を代入する。
2. `ensure_dir(...)` を実行する。
3. `out_csv` に `scenario_dir / 'simulation.csv'` の結果を代入する。
4. `detail_csv` に `scenario_dir / 'simulation_detail.csv'` の結果を代入する。
5. `plan_csv` に `scenario_dir / 'upper_plan.csv'` の結果を代入する。
6. `report_html` に `scenario_dir / 'simulation_report.html'` の結果を代入する。
7. `summary_json` に `scenario_dir / 'summary.json'` の結果を代入する。
8. `resolved_yaml` に `scenario_dir / 'resolved.yaml'` の結果を代入する。
9. `latest_manifest_json` に `scenario_dir / 'latest_manifest.json'` の結果を代入する。
10. `sim_log_path` に `scenario_dir / 'solar_sim_console.log'` の結果を代入する。
11. `cmd` に `[os.fspath(Path(sys.executable)), os.fspath(ROOT / 'scripts' / 'solar_sim.py'), '-profile_yaml', os.fspath(profile_yaml), '-out_csv', os.fspath(out_csv), '-out_detail_csv', os.fspath(detail_csv), '-out_plan_csv', os.fspath(plan_csv), '-report_html', os.fspath(report_html), '-summary_json', os.fspath(summary_json), '-resolved_yaml', os.fspath(resolved_yaml), '-latest_manifest_json', os.fspath(latest_manifest_json)]` の結果を代入する。
12. `merged_cli` に `dict(cli_overrides)` の結果を代入する。
13. `merged_cli.update(...)` を実行する。
14. `merged_cli.items()` を順に走査し、各要素を `(key, value)` に入れて処理する。
15. `cmd.extend(...)` を実行する。
16. `merged_cfg` に `dict(cfg_overrides)` の結果を代入する。
17. `merged_cfg.update(...)` を実行する。
18. `upper_cost.items()` を順に走査し、各要素を `(key, value)` に入れて処理する。
19. `merged_cfg['mpc.upper_cost.{key}']` に `value` の結果を代入する。
20. `merged_cfg['mpc.w_dv']` に `upper_cost.get('w_speed_smooth', 0.0)` の結果を代入する。
21. `merged_cfg['mpc.w_dv_limit']` に `upper_cost.get('w_dv_limit', 0.0)` の結果を代入する。
22. `merged_cfg['mpc.w_speed_limit']` に `upper_cost.get('w_speed_limit', 0.0)` の結果を代入する。
23. `merged_cfg['mpc.w_current']` に `upper_cost.get('w_current_sq', 0.0)` の結果を代入する。
24. `merged_cfg['mpc.w_T']` に `upper_cost.get('w_temp', 0.0)` の結果を代入する。
25. `merged_cfg['mpc.w_soc_day_max']` に `upper_cost.get('w_soc_day_max', 0.0)` の結果を代入する。
26. `merged_cfg['mpc.w_soc_day_track']` に `upper_cost.get('w_soc_day_track', 0.0)` の結果を代入する。
27. `merged_cfg['mpc.w_soc_terminal']` に `upper_cost.get('w_soc_terminal', 0.0)` の結果を代入する。
28. `merged_cfg.items()` を順に走査し、各要素を `(key, value)` に入れて処理する。
29. `cmd.extend(...)` を実行する。
30. `with` 文で `sim_log_path.open('w', encoding='utf-8', newline='\n')` を管理しながら処理する。

31. `proc` に `subprocess.run(cmd, cwd=ROOT, check=False, stdout=log_f, stderr=subprocess.STDOUT, text=True)` の結果を代入する。
32. 条件 `proc.returncode != 0` を判定し、真なら内部処理を行う。
33. `result` に `failed_scenario_result(scenario.name, scenario.weight, upper_cost, error=f'solar_sim failed with exit code {proc.returncode}', cfg_overrides=merged_cfg, cli_overrides=merged_cli, summary_json=summary_json, out_csv=out_csv, detail_csv=detail_csv, plan_csv=plan_csv, report_html=report_html, resolved_yaml=resolved_yaml, sim_log_path=sim_log_path)` の結果を代入する。
34. 上の条件が偽の場合:
35. 例外処理を伴う `try` ブロックを実行する。
36. `summary` に `json.loads(summary_json.read_text(encoding='utf-8'))` の結果を代入する。
37. `sim_df` に `pd.read_csv(out_csv)` の結果を代入する。
38. `detail_df` に `pd.read_csv(detail_csv) if detail_csv.exists() else pd.DataFrame()` の結果を代入する。
39. `result` に `evaluate_simulation(profile_yaml, summary, sim_df, detail_df, upper_cost)` の結果を代入する。
40. `Exception` を捕捉した場合:
41. `result` に `failed_scenario_result(scenario.name, scenario.weight, upper_cost, error=f'postprocess failed: {exc}', cfg_overrides=merged_cfg, cli_overrides=merged_cli, summary_json=summary_json, out_csv=out_csv, detail_csv=detail_csv, plan_csv=plan_csv, report_html=report_html, resolved_yaml=resolved_yaml, sim_log_path=sim_log_path)` の結果を代入する。
42. `result.update(...)` を実行する。
43. `(scenario_dir / 'eval_metrics.json').write_text(...)` を実行する。
44. `result` を返す。

代表コード断片

```
def run_single_scenario(
    profile_yaml: Path,
    output_dir: Path,
    candidate_name: str,
    scenario: ScenarioSpec,
    upper_cost: Dict[str, float],
    cfg_overrides: Dict[str, object],
    cli_overrides: Dict[str, object],
) -> Dict[str, object]:
    scenario_dir = output_dir / "candidates" / candidate_name / scenario.name
    ensure_dir(scenario_dir)

    out_csv = scenario_dir / "simulation.csv"
    detail_csv = scenario_dir / "simulation_detail.csv"
    plan_csv = scenario_dir / "upper_plan.csv"
    report_html = scenario_dir / "simulation_report.html"
    summary_json = scenario_dir / "summary.json"
    resolved_yaml = scenario_dir / "resolved.yaml"
    latest_manifest_json = scenario_dir / "latest_manifest.json"
    sim_log_path = scenario_dir / "solar_sim_console.log"

    cmd = [
        os.fspath(Path(sys.executable)),
        os.fspath(ROOT / "scripts" / "solar_sim.py"),
        "--profile_yaml",
        os.fspath(profile_yaml),
        "--out_csv",
```

```

    os.fspath(out_csv),
    "--out_detail_csv",
    os.fspath(detail_csv),
    "--out_plan_csv",
    os.fspath(plan_csv),
    "--report_html",
    os.fspath(report_html),
    "--summary_json",
    ...

```

7.21 L613 関数 `evaluate_simulation`

- 行範囲: L613–L756
- 所属: module 直下
- 定義: `evaluate_simulation(profile_yaml:Path,summary:Dict[str,object],sim_df:pd.DataFrame,detail_df:pd.DataFrame,upper_cost:Dict[str,float])->Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Series`, `UpperCostConfig`, `abs`, `active_upper_cost_terms`, `astype`, `bool`, `diff`, `exists`, `fillna`, `float`, `from_yaml`, `fspath`
- 戻り値の要点: `{'score': float(score), 'final_distance_km': float(summary['final_distance_km']), 'avg_speed_kmh': float(summary.get('avg_speed_kmh', 0.0)), 'min_soc': min_soc, 'final_soc': final_soc, 'elapsed_hours': elapsed_hours, 'cpu_sec': float(summary.get('cpu_sec', 0.0)), 'finish_reached': finish_reached, 'scenario_feasible': not infeasibility_reasons, 'scenario_feasibility_checks': feasibility_checks, 'scenario_infeasibility_reasons': infeasibility_reasons, 'oscillation_mean_abs_dv_kmh': osc, 'oscillation_p95_abs_dv_kmh': osc_p95, 'current_rms_a': current_rms_a, 'pack_slew_rms_kw': pack_slew_rms_kw, 'high_speed_h': high_speed_h, 'daylight_stop_h': daylight_stop_h, 'daylight_full_soc_h': daylight_full_soc_h, 'unused_finish_soc': unused_finish_soc, 'finish_soc_target': finish_soc_target, 'terminal_soc_error': terminal_soc_error, 'active_term_count': int(len(active_terms)), 'active_terms': active_terms}`
- 主なローカル代入名: `active_pair`, `active_terms`, `current_rms_a`, `current_vals`, `daylight_full_soc_h`, `daylight_mask`, `daylight_stop_h`, `drive_mask`, `dt_hours`, `elapsed_hours`, `feasibility_checks`, `final_soc`, `finish_bonus`, `finish_reached`, `finish_soc_target`, `high_speed_h`, `high_speed_mask`, `infeasibility_reasons`, `min_soc`, `mpc_cfg`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 9、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `speed_vals` に `speed_series(sim_df).to_numpy(dtype=float)` の結果を代入する。
2. 条件 `'time_utc' in sim_df.columns and len(sim_df) >= 2` を判定し、真なら内部処理を行う。
3. `t_series` に `pd.to_datetime(sim_df['time_utc'], format='mixed', utc=True, errors='coerce')` の結果を代入する。
4. `dt_hours` に `t_series.diff().dt.total_seconds().fillna(t_series.diff().dt.total_seconds().median()).fillna(0.0) / 3600.0` の結果を代入する。
5. 上の条件が偽の場合:
6. `t_series` に `pd.Series([pd.NaT] * len(sim_df))` の結果を代入する。
7. `dt_hours` に `pd.Series(np.zeros(len(sim_df), dtype=float))` の結果を代入する。
8. `profile_cfg` に `read_yaml(profile_yaml)` の結果を代入する。
9. `schedule` に `None` の結果を代入する。
10. `schedule_rel` に `((profile_cfg.get('paths', {})) if isinstance(profile_cfg, dict) else {}) or {}).get('drive_schedule_yaml')` の結果を代入する。
11. 条件 `schedule_rel` を判定し、真なら内部処理を行う。
12. `schedule_path` に `(profile_yaml.parent / schedule_rel).resolve()` の結果を代入する。
13. 条件 `schedule_path.exists()` を判定し、真なら内部処理を行う。
14. `schedule` に `DriveSchedule.from_yaml(os.fspath(schedule_path))` の結果を代入する。
15. `daylight_mask` に `sim_df.get('G_poa', pd.Series(np.zeros(len(sim_df), dtype=float))).astype(float) >= 250.0` の結果を代入する。
16. `stopped_mask` に `speed_series(sim_df) <= 1.0` の結果を代入する。
17. `soc_mask` に `sim_df.get('soc', pd.Series(np.zeros(len(sim_df), dtype=float))).astype(float) >= 0.95` の結果を代入する。
18. 条件 `schedule is not None and len(t_series) == len(sim_df)` を判定し、真なら内部処理を行う。
19. `drive_mask` に `t_series.map(lambda ts: bool(pd.notna(ts) and schedule.is_drive_time(ts.to_pydatetime())))` の結果を代入する。
20. 上の条件が偽の場合:
21. `drive_mask` に `pd.Series(np.ones(len(sim_df), dtype=bool))` の結果を代入する。
22. 条件 `len(speed_vals) >= 2` を判定し、真なら内部処理を行う。
23. `active_pair` に `drive_mask.to_numpy(dtype=bool)[1:] & drive_mask.to_numpy(dtype=bool)[: -1] & (speed_vals[1:] > 1.0) & (speed_vals[: -1] > 1.0)` の結果を代入する。
24. `speed_steps` に `np.abs(np.diff(speed_vals))[active_pair]` の結果を代入する。
25. 上の条件が偽の場合:
26. `speed_steps` に `np.zeros(0, dtype=float)` の結果を代入する。
27. `osc` に `float(np.mean(speed_steps)) if len(speed_steps) else 0.0` の結果を代入する。
28. `osc_p95` に `float(np.percentile(speed_steps, 95.0)) if len(speed_steps) else 0.0` の結果を代入する。

29. daylight_stop_h に float(dt_hours[drive_mask & daylight_mask & stopped_mask].sum()) if len(dt_hours) == len(sim_df) else 0.0 の結果を代入する。
30. daylight_full_soc_h に float(dt_hours[drive_mask & daylight_mask & soc_mask].sum()) if len(dt_hours) == len(sim_df) else 0.0 の結果を代入する。
31. high_speed_mask に speed_series(sim_df) >= 85.0 の結果を代入する。
32. high_speed_h に float(dt_hours[drive_mask & high_speed_mask].sum()) if len(dt_hours) == len(sim_df) else 0.0 の結果を代入する。
33. 条件 not detail_df.empty and 'I' in detail_df.columns を判定し、真なら内部処理を行う。
34. current_vals に detail_df['I'].to_numpy(dtype=float) の結果を代入する。
35. current_rms_a に float(np.sqrt(np.mean(np.square(current_vals)))) if len(current_vals) else 0.0 の結果を代入する。
36. 上の条件が偽の場合:
37. current_rms_a に 0.0 の結果を代入する。
38. 条件 not detail_df.empty and 'P_pack' in detail_df.columns and (len(detail_df) >= 2) を判定し、真なら内部処理を行う。
39. pack_slew_kw に np.diff(detail_df['P_pack'].to_numpy(dtype=float)) / 1000.0 の結果を代入する。
40. pack_slew_rms_kw に float(np.sqrt(np.mean(np.square(pack_slew_kw)))) if len(pack_slew_kw) else 0.0 の結果を代入する。
41. 上の条件が偽の場合:
42. pack_slew_rms_kw に 0.0 の結果を代入する。
43. final_soc に float(summary.get('final_soc', 0.0)) の結果を代入する。
44. mpc_cfg に profile_cfg.get('mpc', {}) if isinstance(profile_cfg, dict) else {} の結果を代入する。
45. finish_soc_target に float(mpc_cfg.get('soc_finish_target', 0.12) or 0.12) の結果を代入する。
46. 条件 finish_soc_target <= 0.0 を判定し、真なら内部処理を行う。
47. finish_soc_target に 0.12 の結果を代入する。
48. unused_finish_soc に max(0.0, final_soc - finish_soc_target) の結果を代入する。
49. terminal_soc_error に abs(final_soc - finish_soc_target) の結果を代入する。
50. finish_reached に bool(summary.get('finish_reached', False)) の結果を代入する。
51. min_soc に float(summary.get('min_soc', 0.0)) の結果を代入する。
52. elapsed_hours に float(summary.get('elapsed_hours', 0.0)) の結果を代入する。
53. active_terms に active_upper_cost_terms(UpperCostConfig(**upper_cost), threshold=0.0001) の結果を代入する。
54. finish_bonus に 10000.0 if finish_reached else 0.0 の結果を代入する。
55. score に float(summary['final_distance_km']) + finish_bonus - 2.0 * osc - 1.0 * osc_p95 - 0.2 * current_rms_a - 12.0 * pack_slew_rms_kw - 10.0 * high_speed_h - 40.0 * daylight_stop_h - 100.0 * daylight_full_soc_h - 1000.0 * terminal_soc_error - 0.5 * len(active_terms) - (30.0 * elapsed_hours if finish_reached else 0.0) の結果を代入する。
56. upper_solve_count に int(summary.get('upper_solve_count', 0) or 0) の結果を代入する。

57. `terminal_soc_min` に `float(summary.get('terminal_soc_min', mpc_cfg.get('terminal_soc_min', 0.0)))` の結果を代入する。
58. `feasibility_checks` に `{'finish_reached': finish_reached, 'model_validation_gate_pass': bool(summary.get('model_validation_gate_pass', False)), 'terminal_soc_target_met': bool(summary.get('terminal_soc_target_met', True)), 'minimum_soc_respected': bool(math.isfinite(min_soc) and min_soc >= terminal_soc_min - 1e-06), 'finite_score': bool(math.isfinite(score))}` の結果を代入する。
59. 条件 `upper_solve_count > 0` を判定し、真なら内部処理を行う。
60. `feasibility_checks.update(...)` を実行する。
61. `infeasibility_reasons` に `[name for name, passed in feasibility_checks.items() if not bool(passed)]` の結果を代入する。
62. `{'score': float(score), 'final_distance_km': float(summary['final_distance_km']), 'avg_speed_kmh': float(summary.get('avg_speed_kmh', 0.0)), 'min_soc': min_soc, 'final_soc': final_soc, 'elapsed_hours': elapsed_hours, 'cpu_sec': float(summary.get('cpu_sec', 0.0)), 'finish_reached': finish_reached, 'scenario_feasible': not infeasibility_reasons, 'scenario_feasibility_checks': feasibility_checks, 'scenario_infeasibility_reasons': infeasibility_reasons, 'oscillation_mean_abs_dv_kmh': osc, 'oscillation_p95_abs_dv_kmh': osc_p95, 'current_rms_a': current_rms_a, 'pack_slew_rms_kw': pack_slew_rms_kw, 'high_speed_h': high_speed_h, 'daylight_stop_h': daylight_stop_h, 'daylight_full_soc_h': daylight_full_soc_h, 'unused_finish_soc': unused_finish_soc, 'finish_soc_target': finish_soc_target, 'terminal_soc_error': terminal_soc_error, 'active_term_count': int(len(active_terms)), 'active_terms': active_terms}` を返す。

代表コード断片

```
def evaluate_simulation(
    profile_yaml: Path,
    summary: Dict[str, object],
    sim_df: pd.DataFrame,
    detail_df: pd.DataFrame,
    upper_cost: Dict[str, float],
) -> Dict[str, object]:
    speed_vals = speed_series(sim_df).to_numpy(dtype=float)

    if "time_utc" in sim_df.columns and len(sim_df) >= 2:
        t_series = pd.to_datetime(sim_df["time_utc"], format="mixed", utc=True, errors="coerce")
        dt_hours = t_series.diff().dt.total_seconds().fillna(t_series.diff().dt.total_seconds().median()).fillna(0.0) / 3600.0
    else:
        t_series = pd.Series([pd.NaT] * len(sim_df))
        dt_hours = pd.Series(np.zeros(len(sim_df), dtype=float))

    profile_cfg = read_yaml(profile_yaml)
    schedule = None
    schedule_rel = ((profile_cfg.get("paths", {}) if isinstance(profile_cfg, dict) else {}) or {}).get("drive_schedule_yaml")
    if schedule_rel:
        schedule_path = (profile_yaml.parent / schedule_rel).resolve()
        if schedule_path.exists():
            schedule = DriveSchedule.from_yaml(os.fspath(schedule_path))

    daylight_mask = sim_df.get("G_poa", pd.Series(np.zeros(len(sim_df), dtype=float))).astype(float) >= 250.0
    stopped_mask = speed_series(sim_df) <= 1.0
    soc_mask = sim_df.get("soc", pd.Series(np.zeros(len(sim_df), dtype=float))).astype(float) >= 0.95
    if schedule is not None and len(t_series) == len(sim_df):
        drive_mask = t_series.map(lambda ts: bool(pd.notna(ts) and schedule.is_drive_time(ts.to_pydatetime())))
```



```

else:
    drive_mask = pd.Series(np.ones(len(sim_df), dtype=bool))
    # A scheduled stop/start is not controller chatter. Score speed variation
    # only between consecutive samples that are both actively driving.
    if len(speed_vals) >= 2:
        active_pair = (
...

```

7.22 L759 関数 aggregate_candidate

- 行範囲: L759–L817
- 所属: module 直下
- 定義: `aggregate_candidate(candidate:str, scenario_results:List[Dict[str,object]], weights:Dict[str,float], risk_config:ScenarioRiskConfig|None=None)->Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `aggregate_scenario_scores`, `all`, `array`, `asarray`, `bool`, `dot`, `float`, `get`, `int`, `isfinite`, `len`
- 戻り値の要点: `result`
- 主なローカル代入名: `nominal`, `result`, `risk`, `row`, `scenario_feasible`, `scenario_scores`, `scenario_weights`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('scenario_results is empty')`
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. 条件 `not scenario_results` を判定し、真なら内部処理を行う。
2. `ValueError('scenario_results is empty')` を送出する。
3. `scenario_weights` に `np.array([max(0.0, float(row.get('scenario_weight', 0.0))) for row in scenario_results], dtype=float)` の結果を代入する。
4. 条件 `not np.isfinite(scenario_weights).all() or scenario_weights.sum() <= 0.0` を判定し、真なら内部処理を行う。
5. `scenario_weights` に `np.ones(len(scenario_results), dtype=float)` の結果を代入する。
6. `scenario_scores` に `np.array([float(row['score']) for row in scenario_results], dtype=float)` の結果を代入する。
7. `scenario_feasible` に `np.array([bool(row.get('scenario_feasible', False)) for row in scenario_results], dtype=bool)` の結果を代入する。

8. risk に aggregate_scenario_scores(scenario_scores, scenario_weights, scenario_feasible, config=risk_config) の結果を代入する。
9. scenario_weights に np.asarray(risk['normalized_scenario_weights'], dtype=float) の結果を代入する。
10. nominal に next((row for row in scenario_results if row.get('scenario') == 'nominal'), scenario_results[0]) の結果を代入する。
11. result に {'candidate': candidate, **risk, 'final_distance_km': float(np.dot(scenario_weights, np.array([float(row['final_distance_km']) for row in scenario_results])), 'final_distance_worst_km': float(min((float(row['final_distance_km']) for row in scenario_results))), 'avg_speed_kmh': float(np.dot(scenario_weights, np.array([float(row['avg_speed_kmh']) for row in scenario_results])), 'min_soc': float(min((float(row['min_soc']) for row in scenario_results))), 'final_soc': float(np.dot(scenario_weights, np.array([float(row['final_soc']) for row in scenario_results])), 'finish_reached': bool(all((bool(row['finish_reached']) for row in scenario_results))), 'model_validation_gate_pass_all': bool(all((bool(row.get('model_validation_gate_pass', False)) for row in scenario_results))), 'oscillation_mean_abs_dv_kmh': float(np.dot(scenario_weights, np.array([float(row['oscillation_mean_abs_dv_kmh']) for row in scenario_results])), 'oscillation_p95_abs_dv_kmh': float(np.dot(scenario_weights, np.array([float(row['oscillation_p95_abs_dv_kmh']) for row in scenario_results])), 'current_rms_a': float(np.dot(scenario_weights, np.array([float(row['current_rms_a']) for row in scenario_results])), 'pack_slew_rms_kw': float(np.dot(scenario_weights, np.array([float(row['pack_slew_rms_kw']) for row in scenario_results])), 'high_speed_h': float(np.dot(scenario_weights, np.array([float(row['high_speed_h']) for row in scenario_results])), 'daylight_stop_h': float(np.dot(scenario_weights, np.array([float(row['daylight_stop_h']) for row in scenario_results])), 'daylight_full_soc_h': float(np.dot(scenario_weights, np.array([float(row['daylight_full_soc_h']) for row in scenario_results])), 'unused_finish_soc': float(np.dot(scenario_weights, np.array([float(row['unused_finish_soc']) for row in scenario_results])), 'finish_soc_target': float(np.dot(scenario_weights, np.array([float(row['finish_soc_target']) for row in scenario_results])), 'terminal_soc_error': float(np.dot(scenario_weights, np.array([float(row['terminal_soc_error']) for row in scenario_results])), 'elapsed_hours': float(np.dot(scenario_weights, np.array([float(row['elapsed_hours']) for row in scenario_results])), 'cpu_sec': float(sum((float(row['cpu_sec']) for row in scenario_results))), 'active_term_count': int(round(np.dot(scenario_weights, np.array([float(row['active_term_count']) for row in scenario_results])), 'weights': weights, 'scenario_results': scenario_results, 'nominal_out_csv': nominal.get('out_csv', ''), 'nominal_detail_csv': nominal.get('detail_csv', '')) の結果を代入する。
12. result を返す。

代表コード断片

```
def aggregate_candidate(
    candidate: str,
    scenario_results: List[Dict[str, object]],
    weights: Dict[str, float],
    risk_config: ScenarioRiskConfig | None = None,
) -> Dict[str, object]:
    if not scenario_results:
        raise ValueError("scenario_results is empty")
    scenario_weights = np.array([max(0.0, float(row.get("scenario_weight", 0.0))) for row in
                                scenario_results], dtype=float)
    if not np.isfinite(scenario_weights).all() or scenario_weights.sum() <= 0.0:
        scenario_weights = np.ones(len(scenario_results), dtype=float)
    scenario_scores = np.array([float(row["score"]) for row in scenario_results], dtype=
float)
    scenario_feasible = np.array(
        [bool(row.get("scenario_feasible", False)) for row in scenario_results],
        dtype=bool,
    )
    risk = aggregate_scenario_scores(
        scenario_scores,
        scenario_weights,
        scenario_feasible,
        config=risk_config,
    )
    scenario_weights = np.asarray(risk["normalized_scenario_weights"], dtype=float)
```

```

nominal = next((row for row in scenario_results if row.get("scenario") == "nominal"),
scenario_results[0])
result = {
    "candidate": candidate,
    **risk,
    "final_distance_km": float(np.dot(scenario_weights, np.array([float(row["
final_distance_km"]) for row in scenario_results]))),
    "final_distance_worst_km": float(min(float(row["final_distance_km"]) for row in
scenario_results)),
    "avg_speed_kmh": float(np.dot(scenario_weights, np.array([float(row["avg_speed_kmh"
]) for row in scenario_results]))),
    "min_soc": float(min(float(row["min_soc"]) for row in scenario_results)),
    "final_soc": float(np.dot(scenario_weights, np.array([float(row["final_soc"]) for
row in scenario_results]))),
    "finish_reached": bool(all(bool(row["finish_reached"]) for row in scenario_results))
    ,
    "model_validation_gate_pass_all": bool(
...

```

7.23 L820 関数 run_candidate

- 行範囲: L820–L855
- 所属: module 直下
- 定義: `run_candidate(profile_yaml:Path,output_dir:Path,candidate_name:str,upper_cost:Dict[str,float],cfg_overrides:Dict[str,object],cli_overrides:Dict[str,object],scenarios:List[ScenarioSpec],risk_config:ScenarioRiskConfig|None=None)->Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `aggregate_candidate`, `canonical_runtime_weights`, `dumps`, `ensure_dir`, `run_single_scenario`, `write_text`
- 戻り値の要点: `result`
- 主なローカル代入名: `cand_dir`, `result`, `runtime_upper_cost`, `scenario`, `scenario_results`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `runtime_upper_cost` に `canonical_runtime_weights(upper_cost)` の結果を代入する。
2. `scenario_results` に `[run_single_scenario(profile_yaml,output_dir,candidate_name,scenario,runtime_upper_cost,cfg_overrides,cli_overrides) for scenario in scenarios]` の結果を代入する。

3. result に aggregate_candidate(candidate_name, scenario_results, runtime_upper_cost, risk_config=risk_config) の結果を代入する。
4. cand_dir に output_dir / 'candidates' / candidate_name の結果を代入する。
5. ensure_dir(...) を実行する。
6. (cand_dir / 'aggregate_metrics.json').write_text(...) を実行する。
7. result を返す。

代表コード断片

```
def run_candidate(
    profile_yaml: Path,
    output_dir: Path,
    candidate_name: str,
    upper_cost: Dict[str, float],
    cfg_overrides: Dict[str, object],
    cli_overrides: Dict[str, object],
    scenarios: List[ScenarioSpec],
    risk_config: ScenarioRiskConfig | None = None,
) -> Dict[str, object]:
    runtime_upper_cost = canonical_runtime_weights(upper_cost)
    scenario_results = [
        run_single_scenario(
            profile_yaml,
            output_dir,
            candidate_name,
            scenario,
            runtime_upper_cost,
            cfg_overrides,
            cli_overrides,
        )
        for scenario in scenarios
    ]
    result = aggregate_candidate(
        candidate_name,
        scenario_results,
        runtime_upper_cost,
        risk_config=risk_config,
    )
    cand_dir = output_dir / "candidates" / candidate_name
    ensure_dir(cand_dir)
    (cand_dir / "aggregate_metrics.json").write_text(
        json.dumps(result, indent=2, ensure_ascii=False),
        encoding="utf-8",
    )
    ...
```

7.24 L858 関数 save_trial_checkpoint

- 行範囲: L858–L865
- 所属: module 直下
- 定義: save_trial_checkpoint(output_dir:Path, trials:List[Dict[str,object]], best_result:Dict[str,object]|None)->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: DataFrame, float, get, to_csv, write_yaml

- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `trials` を判定し、真なら内部処理を行う。
2. `pd.DataFrame(trials).to_csv(...)` を実行する。
3. 条件 `best_result is not None` を判定し、真なら内部処理を行う。
4. `write_yaml(...)` を実行する。

代表コード断片

```
def save_trial_checkpoint(output_dir: Path, trials: List[Dict[str, object]], best_result:
    Dict[str, object] | None) -> None:
    if trials:
        pd.DataFrame(trials).to_csv(output_dir / "trial_results_partial.csv", index=False)
    if best_result is not None:
        write_yaml(
            output_dir / "best_upper_cost_partial.yaml",
            {"upper_cost": best_result.get("weights", {}), "score": float(best_result.get(
                "score", 0.0))},
        )
```

7.25 L868 関数 `csv_row_count`

- 行範囲: L868–L870
- 所属: `module` 直下
- 定義: `csv_row_count(path:Path)->int`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `max`, `open`, `sum`
- 戻り値の要点: `max(0, sum((1 for _ in f)) - 1)`
- 主なローカル代入名: `_`, `f`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して **collection** を作る。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. with 文で `path.open('r', encoding='utf-8', errors='ignore')` を管理しながら処理する。
2. `max(0, sum((1 for _ in f)) - 1)` を返す。

代表コード断片

```
def csv_row_count(path: Path) -> int:
    with path.open("r", encoding="utf-8", errors="ignore") as f:
        return max(0, sum(1 for _ in f) - 1)
```

7.26 L873 関数 `summarize_path`

- 行範囲: L873–L893
- 所属: module 直下
- 定義: `summarize_path(path:Path)->Dict[str,object]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `csv_row_count`, `exists`, `fspath`, `join`, `len`, `lower`, `lstrip`, `read_csv`, `str`
- 戻り値の要点: `summary / summary`
- 主なローカル代入名: `col`, `header`, `suffix`, `summary`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して **collection** を作る。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. suffix に path.suffix.lower() の結果を代入する。
2. summary に {'path': os.fspath(path), 'exists': path.exists(), 'kind': suffix.lstrip('.'), 'rows': '', 'columns': '', 'column_names': ''} の結果を代入する。
3. 条件 not path.exists() を判定し、真なら内部処理を行う。
4. summary を返す。
5. 条件 suffix == '.csv' を判定し、真なら内部処理を行う。
6. 例外処理を伴う try ブロックを実行する。
7. header に pd.read_csv(path, nrows=0) の結果を代入する。
8. summary['rows'] に csv_row_count(path) の結果を代入する。
9. summary['columns'] に len(header.columns) の結果を代入する。
10. summary['column_names'] に ', '.join((str(col) for col in header.columns[:12])) の結果を代入する。
11. Exception を捕捉した場合:
12. Pass 文を実行する。
13. summary を返す。

代表コード断片

```
def summarize_path(path: Path) -> Dict[str, object]:
    suffix = path.suffix.lower()
    summary = {
        "path": os.fspath(path),
        "exists": path.exists(),
        "kind": suffix.lstrip("."),
        "rows": "",
        "columns": "",
        "column_names": "",
    }
    if not path.exists():
        return summary
    if suffix == ".csv":
        try:
            header = pd.read_csv(path, nrows=0)
            summary["rows"] = csv_row_count(path)
            summary["columns"] = len(header.columns)
            summary["column_names"] = ", ".join(str(col) for col in header.columns[:12])
        except Exception:
            pass
    return summary
```

7.27 L896 関数 dataframe_to_markdown

- 行範囲: L896–L907
- 所属: module 直下
- 定義: dataframe_to_markdown(df:pd.DataFrame)->str
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: append, iterrows, join, len, replace, str

- 戻り値の要点: `'\n'.join(lines) / '(none)'`
- 主なローカル代入名: `_`, `col`, `cols`, `lines`, `row`, `vals`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. 条件 `df.empty` を判定し、真なら内部処理を行う。
2. `'(none)'` を返す。
3. `cols` に `[str(col) for col in df.columns]` の結果を代入する。
4. `lines` に `[ '|' + ' | '.join(cols) + ' | ', '|' + ' | '.join(['---'] * len(cols)) + ' | ' ]` の結果を代入する。
5. `df.iterrows()` を順に走査し、各要素を `(_, row)` に入れて処理する。
6. `vals` に `[str(row[col]).replace('\n', ' ') for col in df.columns]` の結果を代入する。
7. `lines.append(...)` を実行する。
8. `'\n'.join(lines)` を返す。

代表コード断片

```
def dataframe_to_markdown(df: pd.DataFrame) -> str:
    if df.empty:
        return "(none)"
    cols = [str(col) for col in df.columns]
    lines = [
        "|" + " | ".join(cols) + " |",
        "|" + " | ".join(["---"] * len(cols)) + " |",
    ]
    for _, row in df.iterrows():
        vals = [str(row[col]).replace("\n", " ") for col in df.columns]
        lines.append("| " + " | ".join(vals) + " |")
    return "\n".join(lines)
```

7.28 L910 関数 `flatten_scalars`

- 行範囲: L910–L917
- 所属: `module` 直下
- 定義: `flatten_scalars(prefix:str, payload, rows:List[Dict[str, object]]) -> None`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `flatten_scalars`, `isinstance`, `items`, `str`

- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `key`, `next_prefix`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `isinstance(payload, dict)` を判定し、真なら内部処理を行う。
2. `payload.items()` を順に走査し、各要素を `(key, value)` に入れて処理する。
3. `next_prefix` に `f'{prefix}.{key}' if prefix else str(key)` の結果を代入する。
4. `flatten_scalars(...)` を実行する。
5. を返す。
6. 条件 `isinstance(payload, (int, float, bool, str))` を判定し、真なら内部処理を行う。
7. `rows.append(...)` を実行する。

代表コード断片

```
def flatten_scalars(prefix: str, payload, rows: List[Dict[str, object]]) -> None:
    if isinstance(payload, dict):
        for key, value in payload.items():
            next_prefix = f"{prefix}.{key}" if prefix else str(key)
            flatten_scalars(next_prefix, value, rows)
        return
    if isinstance(payload, (int, float, bool, str)):
        rows.append({"key": prefix, "value": payload})
```

7.29 L920 関数 `build_current_asset_manifests`

- 行範囲: L920–L983
- 所属: module 直下
- 定義: `build_current_asset_manifests(profile_yaml: Path, fit_summary: dict, output_dir: Path) -> Dict[str, Path]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `append`, `dataframe_to_markdown`, `flatten_scalars`, `get`, `isinstance`, `isoformat`, `items`, `join`, `now`, `read_yaml`, `repo_relative`
- 戻り値の要点: `{'files_csv': files_csv, 'scalars_csv': scalars_csv, 'markdown': md_path}`
- 主なローカル代入名: `file_rows`, `files_csv`, `files_df`, `info`, `local_fit_rows`, `local_rows`, `md_lines`, `md_path`, `path`, `paths_cfg`, `profile_cfg`, `rel_path`, `role`, `row`, `scalar_rows`, `scalars_csv`, `scalars_df`, `section`, `section_name`

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 4、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `profile_cfg` に `read_yaml(profile_yaml)` の結果を代入する。
2. `paths_cfg` に `profile_cfg.get('paths', {})` if `isinstance(profile_cfg, dict)` else `{}` の結果を代入する。
3. `file_rows` に `[]` の結果を代入する。
4. `sorted(paths_cfg.items())` を順に走査し、各要素を `(role, rel_path)` に入れて処理する。
5. `path` に `(profile_yaml.parent / str(rel_path)).resolve()` の結果を代入する。
6. `info` に `summarize_path(path)` の結果を代入する。
7. `info['role']` に `role` の結果を代入する。
8. `file_rows.append(...)` を実行する。
9. `files_df` に `pd.DataFrame(file_rows)` の結果を代入する。
10. `files_csv` に `output_dir / 'current_active_files.csv'` の結果を代入する。
11. `files_df.to_csv(...)` を実行する。
12. `scalar_rows` に `[]` を代入する。
13. `('model', 'mpc', 'runtime', 'simulation', 'live', 'measurement')` を順に走査し、各要素を `section_name` に入れて処理する。
14. `section` に `profile_cfg.get(section_name, {})` の結果を代入する。
15. `local_rows` に `[]` を代入する。
16. `flatten_scalars(...)` を実行する。
17. `local_rows` を順に走査し、各要素を `row` に入れて処理する。
18. `scalar_rows.append(...)` を実行する。
19. `local_fit_rows` に `[]` を代入する。
20. `flatten_scalars(...)` を実行する。
21. `local_fit_rows` を順に走査し、各要素を `row` に入れて処理する。
22. `scalar_rows.append(...)` を実行する。
23. `scalars_df` に `pd.DataFrame(scalar_rows)` の結果を代入する。
24. `scalars_csv` に `output_dir / 'current_scalar_coefficients.csv'` の結果を代入する。
25. `scalars_df.to_csv(...)` を実行する。

26. `md_lines` に `['# Current maps and coefficients', '', f'- profile: {repo_relative(profile_yaml)}', f'- generated_at: {datetime.now().isoformat(timespec='seconds')}','', '', '## Active files', '', dataframe_to_markdown(files_df), '', '## Scalar coefficients', '', dataframe_to_markdown(scalars_df), '']` の結果を代入する。
27. `md_path` に `output_dir / 'current_maps_and_coefficients.md'` の結果を代入する。
28. `md_path.write_text(...)` を実行する。
29. `{'files_csv': files_csv, 'scalars_csv': scalars_csv, 'markdown': md_path}` を返す。

代表コード断片

```
def build_current_asset_manifests(profile_yaml: Path, fit_summary: dict, output_dir: Path)
    -> Dict[str, Path]:
    profile_cfg = read_yaml(profile_yaml)
    paths_cfg = profile_cfg.get("paths", {}) if isinstance(profile_cfg, dict) else {}

    file_rows = []
    for role, rel_path in sorted(paths_cfg.items()):
        path = (profile_yaml.parent / str(rel_path)).resolve()
        info = summarize_path(path)
        info["role"] = role
        file_rows.append(info)
    files_df = pd.DataFrame(file_rows)
    files_csv = output_dir / "current_active_files.csv"
    files_df.to_csv(files_csv, index=False)

    scalar_rows: List[Dict[str, object]] = []
    for section_name in ("model", "mpc", "runtime", "simulation", "live", "measurement"):
        section = profile_cfg.get(section_name, {})
        local_rows: List[Dict[str, object]] = []
        flatten_scalars("", section, local_rows)
        for row in local_rows:
            scalar_rows.append(
                {
                    "source": f"profile:{section_name}",
                    "key": row["key"],
                    "value": row["value"],
                }
            )
    local_fit_rows: List[Dict[str, object]] = []
    flatten_scalars("", fit_summary, local_fit_rows)
    for row in local_fit_rows:
        scalar_rows.append(
            {
                "source": "fit_summary",
                "key": row["key"],
                "value": row["value"],
            }
        )
    ...
```

7.30 L986 関数 `render_report`

- 行範囲: L986–L1501
- 所属: module 直下
- 定義: `render_report(output_dir:Path, source_profile_yaml:Path, eval_profile_yaml:Path, fit_summary:dict, baseline_result:Dict[str,object], tuned_result:Dict[str,object], trials_df:pd.DataFrame, best_weights:Dict[str,float], manifest_paths:Dict[str,Path], removed_reference:str, specs:List[TermSpec], search_cfg:Dict[str,object], tensorboard_dir:Path|None)->tuple[Path,Path]`

- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: DataFrame, Path, Series, UpperCostConfig, abs, active_upper_cost_terms, append, arange, as_posix, bar, close, compile_tex
- 戻り値の要点: (tex_path, md_path) / latex_escape(os.path.relpath(path, report_dir)).replace('%', '\\%')
- 主なローカル代入名: _, active_terms, actual_retire_km, base_row, base_scenarios, baseline_series, best_weights_csv, best_weights_csv_label, best_weights_df, estimated_10000_gen_candidates, estimated_10000_gen_cpu_days, eval_profile_label, files_csv_label, fit_csv, fit_df, fit_rows, human_gap_km, inactive_terms, item, iter_compare_csv
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 7、ループ 4、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して collection を作る。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. score_label に ‘robust score’ if str(search_cfg.get(‘scenario_mode’, ‘nominal’)) == ‘robust’ else ‘aggregate score’ の結果を代入する。
2. report_dir に output_dir / ‘report’ の結果を代入する。
3. ensure_dir(...) を実行する。
4. source_profile_label に repo_relative(source_profile_yaml) の結果を代入する。
5. eval_profile_label に repo_relative(eval_profile_yaml) の結果を代入する。
6. removed_reference_label に repo_relative(removed_reference) or ‘(none)’ の結果を代入する。
7. files_csv_label に repo_relative(manifest_paths[‘files_csv’]) の結果を代入する。
8. scalars_csv_label に repo_relative(manifest_paths[‘scalars_csv’]) の結果を代入する。
9. markdown_label に repo_relative(manifest_paths[‘markdown’]) の結果を代入する。
10. best_weights_csv_label に repo_relative(output_dir / ‘best_upper_cost.csv’) の結果を代入する。
11. tensorboard_label に repo_relative(tensorboard_dir) if tensorboard_dir else ” の結果を代入する。
12. actual_retire_km に float(fit_summary.get(‘race_distance’, {})).get(‘actual_retire_km’, 2831.0)) の結果を代入する。
13. power_rmse_fit に float(fit_summary.get(‘validation_metrics’, {})).get(‘power_rmse_fit_window_w’, fit_summary.get(‘validation_metrics’, {})).get(‘power_rmse_clean_w’, float(‘nan’))) の結果を代入する。
14. voltage_rmse_fit に float(fit_summary.get(‘validation_metrics’, {})).get(‘voltage_rmse_fit_window_v’, fit_summary.get(‘validation_metrics’, {})).get(‘voltage_rmse_clean_v’, float(‘nan’))) の結果を代入する。

15. `median_cpu_sec` に `float(pd.to_numeric(trials_df.get('cpu_sec', pd.Series(dtype=float)), errors='coerce').dropna().median() if not trials_df.empty else float('nan'))` の結果を代入する。
16. `search_candidates` に `int(sum((1 for _, row in trials_df.iterrows() if str(row.get('candidate', '')).startswith('g'))))` の結果を代入する。
17. `estimated_10000_gen_candidates` に `int(search_cfg.get('population', 0)) * 10000` の結果を代入する。
18. `estimated_10000_gen_cpu_days` に `median_cpu_sec * estimated_10000_gen_candidates / 86400.0 if math.isfinite(median_cpu_sec) and median_cpu_sec > 0.0 and (estimated_10000_gen_candidates > 0) else float('nan')` の結果を代入する。
19. `human_gap_km` に `actual_retire_km - float(tuned_result['final_distance_km'])` の結果を代入する。
20. `learning_png` に `report_dir / 'learning_curve.png'` の結果を代入する。
21. 条件 `not trials_df.empty` を判定し、真なら内部処理を行う。
22. `plt.figure(...)` を実行する。
23. `plt.plot(...)` を実行する。
24. `plt.plot(...)` を実行する。
25. `plt.xlabel(...)` を実行する。
26. `plt.ylabel(...)` を実行する。
27. `plt.title(...)` を実行する。
28. `plt.grid(...)` を実行する。
29. `plt.legend(...)` を実行する。
30. `plt.tight_layout(...)` を実行する。
31. `plt.savefig(...)` を実行する。
32. `plt.close(...)` を実行する。
33. `scenario_compare_png` に `report_dir / 'scenario_compare.png'` の結果を代入する。
34. `base_scenarios` に `{row['scenario']: row for row in baseline_result['scenario_results']}` の結果を代入する。
35. `tuned_scenarios` に `{row['scenario']: row for row in tuned_result['scenario_results']}` の結果を代入する。
36. `scenario_names` に `list(base_scenarios.keys())` の結果を代入する。
37. `x` に `np.arange(len(scenario_names))` の結果を代入する。
38. `width` に `0.35` の結果を代入する。
39. `plt.figure(...)` を実行する。
40. `plt.bar(...)` を実行する。
41. `plt.bar(...)` を実行する。
42. `plt.xticks(...)` を実行する。
43. `plt.ylabel(...)` を実行する。
44. `plt.title(...)` を実行する。
45. `plt.grid(...)` を実行する。

46. plt.legend(...) を実行する。
47. plt.tight_layout(...) を実行する。
48. plt.savefig(...) を実行する。
49. plt.close(...) を実行する。
50. baseline_series に pd.read_csv(os.fspath(baseline_result['nominal_out_csv'])) の結果を代入する。
51. tuned_series に pd.read_csv(os.fspath(tuned_result['nominal_out_csv'])) の結果を代入する。
52. speed_compare_png に report_dir / 'speed_compare.png' の結果を代入する。
53. plt.figure(...) を実行する。
54. plt.plot(...) を実行する。
55. plt.plot(...) を実行する。
56. plt.xlabel(...) を実行する。
57. plt.ylabel(...) を実行する。
58. plt.title(...) を実行する。
59. plt.grid(...) を実行する。
60. plt.legend(...) を実行する。
61. plt.tight_layout(...) を実行する。
62. plt.savefig(...) を実行する。
63. plt.close(...) を実行する。
64. soc_compare_png に report_dir / 'soc_compare.png' の結果を代入する。
65. plt.figure(...) を実行する。
66. plt.plot(...) を実行する。
67. plt.plot(...) を実行する。
68. plt.xlabel(...) を実行する。
69. plt.ylabel(...) を実行する。
70. plt.title(...) を実行する。
71. plt.grid(...) を実行する。
72. plt.legend(...) を実行する。
73. plt.tight_layout(...) を実行する。
74. plt.savefig(...) を実行する。
75. plt.close(...) を実行する。
76. best_weights_df に pd.DataFrame({'term': list(best_weights.keys()), 'value': [float(best_weights[key]) for key in best_weights]}) の結果を代入する。
77. best_weights_csv に output_dir / 'best_upper_cost.csv' の結果を代入する。
78. best_weights_df.to_csv(...) を実行する。
79. scenario_rows に [] の結果を代入する。
80. baseline_result['scenario_results'] を順に走査し、各要素を base_row に入れて処理する。

代表コード断片

```
def render_report(
    output_dir: Path,
    source_profile_yaml: Path,
    eval_profile_yaml: Path,
    fit_summary: dict,
    baseline_result: Dict[str, object],
    tuned_result: Dict[str, object],
    trials_df: pd.DataFrame,
    best_weights: Dict[str, float],
    manifest_paths: Dict[str, Path],
    removed_reference: str,
    specs: List[TermSpec],
    search_cfg: Dict[str, object],
    tensorboard_dir: Path | None,
) -> tuple[Path, Path]:
    score_label = "robust score" if str(search_cfg.get("scenario_mode", "nominal")) == "
robust" else "aggregate score"
    report_dir = output_dir / "report"
    ensure_dir(report_dir)
    source_profile_label = repo_relative(source_profile_yaml)
    eval_profile_label = repo_relative(eval_profile_yaml)
    removed_reference_label = repo_relative(removed_reference) or "(none)"
    files_csv_label = repo_relative(manifest_paths["files_csv"])
    scalars_csv_label = repo_relative(manifest_paths["scalars_csv"])
    markdown_label = repo_relative(manifest_paths["markdown"])
    best_weights_csv_label = repo_relative(output_dir / "best_upper_cost.csv")
    tensorboard_label = repo_relative(tensorboard_dir) if tensorboard_dir else ""
    actual_retire_km = float(fit_summary.get("race_distance", {}).get("actual_retire_km",
2831.0))
    power_rmse_fit = float(
        fit_summary.get("validation_metrics", {}).get(
            "power_rmse_fit_window_w",
            fit_summary.get("validation_metrics", {}).get("power_rmse_clean_w", float("nan"))
        ),
    )
    voltage_rmse_fit = float(
        fit_summary.get("validation_metrics", {}).get(
...

```

7.31 L1138 関数 render_report.tex_path_rel

- 行範囲: L1138–L1139
- 所属: render_report
- 定義: tex_path_rel(path:Path)->str
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: latex_escape, relpath, replace
- 戻り値の要点: latex_escape(os.path.relpath(path, report_dir)).replace('%', '\\%')
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `latex_escape(os.path.relpath(path, report_dir)).replace('%', '\\%')` を返す。

代表コード断片

```
def tex_path_rel(path: Path) -> str:
    return latex_escape(os.path.relpath(path, report_dir)).replace("%", r"%")
```

7.32 L1504 関数 main

- 行範囲: L1504–L1937
- 所属: module 直下
- 定義: `main()->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `DataFrame`, `Path`, `RuntimeError`, `ScenarioRiskConfig`, `SummaryWriter`, `UpperCostConfig`, `add_argument`, `append`, `array`, `bool`, `build_current_asset_manifests`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `ap`, `args`, `backup_profile`, `base_cost_cfg`, `base_cost_seed`, `base_weights`, `baseline_exact`, `best_result`, `best_weights_yaml`, `coarse_candidate`, `coarse_cfg_overrides`, `coarse_cli_overrides`, `coarse_result`, `coarse_trials`, `correlated_grid`, `dotted_key`, `elite`, `elite_vecs`, `eval_profile_yaml`, `fit_summary`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `RuntimeError('No candidate was evaluated.')`, `RuntimeError('Validation stage produced no result.')`
- 制御構造の規模: 条件分岐 15、ループ 6、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `ap` に `argparse.ArgumentParser()` の結果を代入する。
2. `ap.add_argument(...)` を実行する。
3. `ap.add_argument(...)` を実行する。
4. `ap.add_argument(...)` を実行する。
5. `ap.add_argument(...)` を実行する。

6. `ap.add_argument(...)` を実行する。
7. `ap.add_argument(...)` を実行する。
8. `ap.add_argument(...)` を実行する。
9. `ap.add_argument(...)` を実行する。
10. `ap.add_argument(...)` を実行する。
11. `ap.add_argument(...)` を実行する。
12. `ap.add_argument(...)` を実行する。
13. `ap.add_argument(...)` を実行する。
14. `ap.add_argument(...)` を実行する。
15. `ap.add_argument(...)` を実行する。
16. `ap.add_argument(...)` を実行する。
17. `ap.add_argument(...)` を実行する。
18. `ap.add_argument(...)` を実行する。
19. `ap.add_argument(...)` を実行する。
20. `ap.add_argument(...)` を実行する。
21. `args` に `ap.parse_args()` の結果を代入する。
22. `profile_yaml` に `Path(args.profile_yaml).resolve()` の結果を代入する。
23. `profile_cfg` に `read_yaml(profile_yaml)` の結果を代入する。
24. `base_cost_cfg` に `load_upper_cost_config(profile_cfg.get('mpc', {}), legacy=profile_cfg.get('mpc', {}))` の結果を代入する。
25. `scenario_mode` に `str(args.scenario_mode).strip().lower()` の結果を代入する。
26. `risk_config` に `ScenarioRiskConfig(cvar_alpha=float(args.risk_cvar_alpha), cvar_weight=float(args.risk_cvar_weight), variance_weight=float(args.risk_variance_weight), max_failure_probability=float(args.max_scenario_failure_probability), infeasible_score=float(args.infeasible_score))` の結果を代入する。
27. `include_terminal_term` に `float(profile_cfg.get('mpc', {})).get('soc_finish_target', -1.0)) > 0.0` の結果を代入する。
28. `include_uncertainty_term` に `scenario_mode != 'nominal'` の結果を代入する。
29. `base_cost_seed` に `base_cost_cfg.to_dict()` の結果を代入する。
30. 条件 `not include_uncertainty_term` を判定し、真なら内部処理を行う。
31. `base_cost_seed['w_uncertainty_reserve']` に `0.0` の結果を代入する。
32. `base_cost_seed['reserve_soc_per_hour']` に `0.0` の結果を代入する。
33. `base_cost_seed['reserve_soc_max_extra']` に `0.0` の結果を代入する。
34. 条件 `not include_terminal_term` を判定し、真なら内部処理を行う。
35. `base_cost_seed['w_soc_terminal']` に `0.0` の結果を代入する。
36. `base_cost_cfg` に `UpperCostConfig(**base_cost_seed)` の結果を代入する。

37. specs に upper_cost_specs(base_cost_cfg, include_progress_terms=args.include_progress_terms, include_uncertainty_term=include_uncertainty_term, include_terminal_term=include_terminal_term) の結果を代入する。
38. rng に np.random.default_rng(args.seed) の結果を代入する。
39. planning_race_km に float(profile_cfg.get('mpc', {}).get('race_km', 3035.5)) の結果を代入する。
40. package_dir に profile_yaml.parent の結果を代入する。
41. timestamp に datetime.now().strftime('%Y%m%d_%H%M%S') の結果を代入する。
42. output_dir に package_dir / 'outputs' / 'self_learning_upper' / timestamp の結果を代入する。
43. ensure_dir(...) を実行する。
44. tensorboard_dir に output_dir / 'tensorboard' の結果を代入する。
45. tb_writer に SummaryWriter(log_dir=os.fspath(tensorboard_dir)) if SummaryWriter is not None else None の結果を代入する。
46. (eval_profile_yaml, removed_reference) に build_reference_free_profile(profile_yaml, output_dir, disable_uncertainty_reserve=scenario_mode == 'nominal') の結果を代入する。
47. fit_summary_path に package_dir / 'outputs' / 'identification' / f'{package_dir.name}_fit_summary.yaml' の結果を代入する。
48. fit_summary に read_yaml(fit_summary_path) if fit_summary_path.exists() else {} の結果を代入する。
49. manifest_paths に build_current_asset_manifests(profile_yaml, fit_summary, output_dir) の結果を代入する。
50. coarse_cfg_overrides に {'mpc.dt': 5400.0, 'mpc.upper_horizon_mode': 'adaptive_full_race', 'mpc.upper_max_iter': int(args.coarse_upper_max_iter), 'mpc.upper_max_steps': 6, 'mpc.race_km': planning_race_km, 'mpc.upper_horizon_km': planning_race_km, 'mpc.upper_ctrl_km': 700.0, 'mpc.upper_replan_km': 0.0, 'mpc.upper_replan_sec': 0.0, 'mpc.upper_adaptive_min_ds_km': 20.0, 'mpc.upper_adaptive_max_ds_km': 400.0, 'mpc.upper_adaptive_growth': 1.3, 'mpc.reference_speed_tracking.enabled': False, 'mpc.upper_cost.w_uncertainty_reserve': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.w_uncertainty_reserve), 'mpc.upper_cost.reserve_soc_per_hour': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.reserve_soc_per_hour), 'mpc.upper_cost.reserve_soc_max_extra': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.reserve_soc_max_extra)} の結果を代入する。
51. coarse_cli_overrides に {} の結果を代入する。
52. validation_cfg_overrides に {'mpc.dt': 2400.0, 'mpc.upper_horizon_mode': 'adaptive_full_race', 'mpc.upper_max_iter': int(args.validation_upper_max_iter), 'mpc.upper_max_steps': 10, 'mpc.race_km': planning_race_km, 'mpc.upper_horizon_km': planning_race_km, 'mpc.upper_ctrl_km': 500.0, 'mpc.upper_replan_km': 0.0, 'mpc.upper_replan_sec': 0.0, 'mpc.upper_adaptive_min_ds_km': 20.0, 'mpc.upper_adaptive_max_ds_km': 350.0, 'mpc.upper_adaptive_growth': 1.28, 'mpc.reference_speed_tracking.enabled': False, 'mpc.upper_cost.w_uncertainty_reserve': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.w_uncertainty_reserve), 'mpc.upper_cost.reserve_soc_per_hour': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.reserve_soc_per_hour), 'mpc.upper_cost.reserve_soc_max_extra': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.reserve_soc_max_extra)} の結果を代入する。
53. validation_cli_overrides に {} の結果を代入する。
54. medium_cfg_overrides に {'mpc.dt': 3600.0, 'mpc.upper_horizon_mode': 'adaptive_full_race', 'mpc.upper_max_iter': int(args.elite_medium_upper_max_iter), 'mpc.upper_max_steps': 8, 'mpc.race_km': planning_race_km, 'mpc.upper_horizon_km': planning_race_km, 'mpc.upper_ctrl_km': 600.0, 'mpc.upper_replan_km': 0.0, 'mpc.upper_replan_sec': 0.0, 'mpc.upper_adaptive_min_ds_km': 20.0, 'mpc.upper_adaptive_max_ds_km': 380.0, 'mpc.upper_adaptive_growth': 1.29, 'mpc.reference_speed_tracking.enabled': False, 'mpc.upper_cost.w_uncertainty_reserve': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.w_uncertainty_reserve), 'mpc.upper_cost.reserve_soc_per_hour': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.reserve_soc_per_hour), 'mpc.upper_cost.reserve_soc_max_extra': 0.0 if scenario_mode == 'nominal' else float(base_cost_cfg.reserve_soc_max_extra)} の結果を代入する。

55. `medium_cli_overrides` に `{}` の結果を代入する。
56. 条件 `args.fidelity_mode == 'correlated'` を判定し、真なら内部処理を行う。
57. `correlated_grid` に `{'mpc.dt': 2400.0, 'mpc.upper_max_steps': 10, 'mpc.upper_ctrl_km': 500.0, 'mpc.upper_adaptive_min': 20.0, 'mpc.upper_adaptive_max_ds_km': 350.0, 'mpc.upper_adaptive_growth': 1.28}` の結果を代入する。
58. `coarse_cfg_overrides.update(...)` を実行する。
59. `medium_cfg_overrides.update(...)` を実行する。
60. `validation_cfg_overrides.update(...)` を実行する。
61. `scenarios` に `default_scenarios(eval_profile_yaml, mode=scenario_mode)` の結果を代入する。
62. `base_weights` に `base_cost_cfg.to_dict()` の結果を代入する。
63. 条件 `scenario_mode == 'nominal'` を判定し、真なら内部処理を行う。
64. `base_weights['w_uncertainty_reserve']` に `0.0` の結果を代入する。
65. `base_weights['reserve_soc_per_hour']` に `0.0` の結果を代入する。
66. `base_weights['reserve_soc_max_extra']` に `0.0` の結果を代入する。
67. `baseline_exact` に `run_candidate(eval_profile_yaml, output_dir, 'validation_baseline', base_weights, validation_cfg_overrides, validation_cli_overrides, scenarios, risk_config)` の結果を代入する。
68. `baseline_exact['generation']` に `-1` の結果を代入する。
69. `baseline_exact['trial_index']` に `-1` の結果を代入する。
70. `log_trial_to_tensorboard(...)` を実行する。
71. `mean` に `np.array([spec.init_log10 for spec in specs], dtype=float)` の結果を代入する。
72. `sigma` に `np.array([0.75] * len(specs), dtype=float)` の結果を代入する。
73. `trials` に `[]` の結果を代入する。
74. `best_result` に `None` の結果を代入する。
75. `trial_index` に `0` の結果を代入する。
76. `range(args.generations)` を順に走査し、各要素を `generation` に入れて処理する。
77. `generation_results` に `[]` の結果を代入する。
78. `range(args.population)` を順に走査し、各要素を `pop_idx` に入れて処理する。
79. 条件 `generation == 0 and pop_idx == 0` を判定し、真なら内部処理を行う。
80. `vec` に `mean.copy()` の結果を代入する。

代表コード断片

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--profile_yaml", default=os.fspath(DEFAULT_PROFILE))
    ap.add_argument("--output_profile_yaml", default="")
    ap.add_argument("--generations", type=int, default=16)
    ap.add_argument("--population", type=int, default=8)
    ap.add_argument("--elite_count", type=int, default=3)
    ap.add_argument("--validation_top_k", type=int, default=5)
    ap.add_argument("--elite_medium_top_k", type=int, default=4)
    ap.add_argument("--seed", type=int, default=7)
    ap.add_argument("--include_progress_terms", action="store_true")
```

```

ap.add_argument("--scenario-mode", choices=["nominal", "robust"], default="nominal")
ap.add_argument("--coarse-upper-max-iter", type=int, default=1)
ap.add_argument("--elite-medium-upper-max-iter", type=int, default=3)
ap.add_argument("--validation-upper-max-iter", type=int, default=8)
ap.add_argument("--risk-cvar-alpha", type=float, default=0.90)
ap.add_argument("--risk-cvar-weight", type=float, default=0.30)
ap.add_argument("--risk-variance-weight", type=float, default=0.0)
ap.add_argument("--max-scenario-failure-probability", type=float, default=0.01)
ap.add_argument("--infeasible-score", type=float, default=-1.0e9)
ap.add_argument(
    "--fidelity-mode",
    choices=["correlated", "legacy_fast"],
    default="correlated",
    help="Use identical discretization across fidelities so candidate ranking remains
valid.",
)
args = ap.parse_args()

profile_yaml = Path(args.profile_yaml).resolve()
profile_cfg = read_yaml(profile_yaml)
base_cost_cfg = load_upper_cost_config(profile_cfg.get("mpc", {}), legacy=profile_cfg.get("mpc", {}))
scenario_mode = str(args.scenario_mode).strip().lower()
risk_config = ScenarioRiskConfig(
    cvar_alpha=float(args.risk_cvar_alpha),
    cvar_weight=float(args.risk_cvar_weight),
...

```

7.33 CLI 引数

行	引数
1506	--profile_yaml
1507	--output_profile_yaml
1508	--generations
1509	--population
1510	--elite_count
1511	--validation_top_k
1512	--elite_medium_top_k
1513	--seed
1514	--include_progress_terms
1515	--scenario-mode
1516	--coarse-upper-max-iter
1517	--elite-medium-upper-max-iter
1518	--validation-upper-max-iter
1519	--risk-cvar-alpha
1520	--risk-cvar-weight
1521	--risk-variance-weight
1522	--max-scenario-failure-probability
1523	--infeasible-score
1524	--fidelity-mode

8 処理の流れ

1. CLI 引数を解釈し、`main()` から処理を起動する。

9 このファイルの位置づけ

この資料群では、`scripts/tune_upper_planner_weights.py` を **planning research** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。